PUCRS

ESCOLA POLITÉCNICA
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO
DOUTORADO EM CIÊNCIA DA COMPUTAÇÃO

ALEXANDRE YUKIO ICHIDA

A BDI-BASED ARCHITECTURE FOR AUTONOMOUS AGENTS IN TEXTUAL ENVIRONMENTS

Porto Alegre
2025

PÓS-GRADUAÇÃO - *STRICTO SENSU*

Pontifícia Universidade Católica
do Rio Grande do Sul

PONTIFICAL CATHOLIC UNIVERSITY OF RIO GRANDE DO SUL
SCHOOL OF TECHNOLOGY
COMPUTER SCIENCE GRADUATE PROGRAM

A BDI-BASED ARCHITECTURE FOR AUTONOMOUS AGENTS IN TEXTUAL ENVIRONMENTS

ALEXANDRE YUKIO ICHIDA

Doctoral Thesis submitted to the Pontifical Catholic University of Rio Grande do Sul in partial fulfillment of the requirements for the degree of Ph. D. in Computer Science.

Advisor: Prof. Dr. Felipe Rech Meneguzzi

Porto Alegre
2025

Ficha Catalográfica

I16b Ichida, Alexandre Yukio

A BDI-Based Architecture for Autonomous Agents in Textual Environments / Alexandre Yukio Ichida. – 2025.
95.
Tese (Doutorado) – Programa de Pós-Graduação em Ciência da Computação, PUCRS.

Orientador: Prof. Dr. Felipe Rech Meneguzzi.

1. belief-desire-intention. 2. processamento de linguagem natural. 3. aprendizado de máquina. 4. agentes autônomos. 5. ambientes textuais. I. Meneguzzi, Felipe Rech. II. Título.

Elaborada pelo Sistema de Geração Automática de Ficha Catalográfica da PUCRS
com os dados fornecidos pelo(a) autor(a).
Bibliotecária responsável: Clarissa Jesinska Selbach CRB-10/2051

ALEXANDRE YUKIO ICHIDA

A BDI-BASED ARCHITECTURE FOR AUTONOMOUS AGENTS IN TEXTUAL ENVIRONMENTS

This Doctoral Thesis has been submitted in partial fulfillment of the requirements for the degree of Ph. D. in Computer Science of the Computer Science Graduate Program, School of Technology of the Pontifical Catholic University of Rio Grande do Sul

Sanctioned on March 25, 2025.

COMMITTEE MEMBERS:

Prof. Dr. Duncan Dubugras Alcoba Ruiz (PPGCC/PUCRS)

Prof. Dr. Alessandro Ricci (Università di Bologna)

Prof. Dr. Lavindra De Silva (Cambridge University)

Prof. Dr. Felipe Rech Meneguzzi (PPGCC/PUCRS - Advisor)

ACKNOWLEDGMENTS

I would like to acknowledge my wife, Jessica Rodrigues da Silva, for all her support throughout my life during the entire PhD program. I would like to express my gratitude to my advisor, Felipe Meneguzzi, for all his guidance, lessons in writing, insights, and ideas throughout this PhD journey, as well as for being a tourist guide in Aberdeen for my wife and me. I am also grateful to Rafael C. Cardoso for working with me on an accepted paper, presenting our work at AAMAS-2024, and for his time helping me review this thesis by sharing his thoughts and insights. Finally, I would like to thank the anonymous reviewers of AAMAS-2024 for providing feedback on our papers, which allowed us to improve this work.

“Once you know what failure feels like, determination chases success.”
(Kobe Bryant)

ARQUITETURA DE AGENTES AUTÔNOMOS PARA AMBIENTES TEXTUAIS BASEADA EM BDI

RESUMO

Desenvolver agentes autônomos para lidar com problemas do mundo real é desafiador, principalmente quando os desenvolvedores não são necessariamente especialistas em inteligência artificial. Avanços recentes em aprendizado de máquina para lidar com tarefas de processamento de linguagem natural estão atingindo níveis de desempenho relevantes para aplicações práticas, embora essas abordagens dependam de modelos opacos e obscuros. Esses fatores impõe três desafios em relação à interface da programação com o desenvolvedor, a eficiência dos agentes resultantes e a avaliação de seu comportamento. Nesta tese, abordamos esses desafios por meio de uma arquitetura eficiente de agentes que aproveita os desenvolvimentos recentes em processamento de linguagem natural e a abstração intuitiva da psicologia popular representada pela arquitetura de beliefs, desires, intentions (BDI). A arquitetura resultante lida com ambientes de linguagem natural utilizando técnicas existentes de aprendizado de máquina para aprimorar as capacidades de raciocínio do agente, ao mesmo tempo em que permite que um desenvolvedor o instrua de forma mais direta, usando a linguagem natural como sua interface de programação. Demonstramos empiricamente os ganhos de eficiência dessa combinação, ao mesmo tempo em que introduzimos um agente autônomo transparente e compreensível para humanos.

Palavras-Chave: belief-desire-intention, processamento de linguagem natural, aprendizado de máquina, agentes autônomos, ambientes textuais.

A BDI-BASED ARCHITECTURE FOR AUTONOMOUS AGENTS IN TEXTUAL ENVIRONMENTS

ABSTRACT

Developing autonomous agents to deal with real-world problems is challenging, especially when developers are not necessarily specialists in artificial intelligence. Recent advances in machine learning to address natural language processing tasks are reaching performance levels suitable for practical applications, although these approaches rely on opaque and inscrutable models. This poses three key challenges regarding the interface of the programming with the developer, the efficiency of the resulting agents, and the scrutiny of their behaviour. In this thesis, we tackle these challenges in an efficient agent architecture that leverages recent developments in natural language processing, and the intuitive folk psychology abstraction of the beliefs, desires, intentions (BDI) architecture. The resulting architecture handles natural language environments using existing machine learning techniques to bootstrap the agent's reasoning capabilities while allowing a developer to instruct the agent more directly using natural language as its programming interface. We empirically demonstrate the efficiency gains of this combination while introducing a transparent and human-understandable autonomous agent.

Keywords: belief-desire-intention, natural language processing, machine learning, autonomous agents, text-based environments.

LIST OF FIGURES

Figure 2.1 – Diagram illustrating a simple form of the BDI agent architecture and its components. 22
Figure 2.2 – Example of Science World task with the state encoded in natural language and the action *open door to the hallway* selected. 32
Figure 2.3 – Graph representing a neural network model and its layers. 34
Figure 2.4 – Diagram of operations executed by self-attention mechanism, generating the representation of the word “thinking” given the input sentence “Thinking Machine”. The numbers are hypothetical. 36
Figure 2.5 – Diagram of self-attention model using Multi-Head approach using 3 heads. 37
Figure 3.1 – Diagram illustrating the NatBDI architecture to handle and actuate over natural language environments. 44
Figure 3.2 – Image comparing the belief base representation by using symbolic beliefs in AgentSpeak language and natural language beliefs by using our approach. Grounded predicates in this image are exclusively for illustrative purposes. 46
Figure 3.3 – NatBDI reasoning cycle example with plans addressing the ScienceWorld “melt” task. 57
Figure 4.1 – Matrix M consisting of every belief-plan context pair to be inferred. . 60
Figure 4.2 – Matrix E with the boolean results of NLI model inference for each belief-context sentence pair. 60
Figure 4.3 – Boolean operation performed to check if the belief base entails the plan context. 61
Figure 4.4 – Graph with rooms obtained from random explorations in the ScienceWorld environment. 64
Figure 4.5 – Scores per episodes when scaling the number of plan-rules and the number of training episodes. 70

LIST OF TABLES

Table 4.1 – Examples of variations for the ScienceWorld task *find-non-living-thing*. 62

Table 4.2 – Examples of variations for the ScienceWorld task *melt*. 63

Table 4.3 – ScienceWorld results from NatBDI execution using two different fallback policy types: Swift and DRRN. We report the results by detailing the total scores, the scores from plans (BDI), and those from fallback actions (FB), as well as the average number of actions executed from plans (BDI) and from the fallback policy (FB). We highlight which approach that contributes more in the total score in bold. 67

Table 4.4 – Experimental results of logical inferences from using different NLI models in NatBDI across all "melt" task variation. We show the best model, the smaller model and the best results in bold. The following columns describe the model name, number of parameters (in millions), the MNLI accuracy on dev set, the score, the average number of actions performed, total plan executions, total missed plan executions and its percentage. 71

Table 4.5 – Experimental results of our lexical analysis on all entailment sentence pairs. We report the average lexical overlap, the number of pairs, and the average word count in beliefs and plan contexts. 73

Table 6.1 – Inventory of semantic relations of MacCartney *et al.* [41] extension of Natural Logic 80

LIST OF ALGORITHMS

Algorithm 2.1 – Abstract implementation of a BDI interpreter [43]. 25
Algorithm 2.2 – Function that updates the event queue given perceptions obtained from environment [43]. 25
Algorithm 2.3 – Function to update belief base according to the belief addition/deletion events perceived in event queue [43]. 26
Algorithm 2.4 – Function that queries the plan library and returns the candidate plan [43]. 26
Algorithm 2.5 – Function to select plans and fill the intention structure [43]. 27
Algorithm 2.6 – Function to add new intentions in the agent intention structure [43]. 27
Algorithm 2.7 – Function to instantiate intentions given a new intention stack with all substitutions applied [43]. 27
Algorithm 2.8 – Function to instantiate intentions given a new intention stack with all substitutions applied [43]. 28
Algorithm 3.1 – Abstract implementation of the NatBDI interpreter based on the BDI interpreter from Algorithm 2.1 52
Algorithm 3.2 – Option Selection algorithm adapted from Algorithm 2.4 to select natural language plans from the NatBDI's plan library, ... 54
Algorithm 3.3 – Plan selection algorithm adapted from Algorithm 2.5 to retrieve candidate options from the NatBDI's plan library. 55
Algorithm 3.4 – Function to instantiate intentions given a new intention stack with all substitutions applied [43]. 56
Algorithm 3.5 – Fallback policy function that includes the fallback action a_{fb} into the intention structure $\mathcal{I}$. 56

LIST OF ACRONYMS

BDI – Belief-Desire-Intention

NLI – Natural Language Inference

MNLI – Multi-Genre Natural Language Inference

LLM – Large Language Models

RL – Reinforcement Learning

ML – Machine Learning

LLM – Large Language Models

NLP – Natural Language Processing

BRF – Belief Revision Function

MDP – Markov Decision Process

POMDP – Partially Observable Markov Decision Process

DRRN – Deep Reinforcement Relevance Network

TD – Task Description

LIST OF SYMBOLS

e – Event 15
φ – Belief 15
$\mathcal{B}$ – Belief Base 15
$\mathcal{PL}$ – Plan Library 15
$\mathcal{P}$ – Plan Rule 15
$\mathcal{C}$ – Plan Context 15
$\mathcal{E}$ – Event Queue 15
$\mathcal{I}$ – Intention Structure 15
π_{FB} – Fallback Policy 15
Pr – Transition Model 15
γ – Discount Rate 15
$\mathcal{S}$ – States 15
$\mathcal{A}$ – Actions 15
$\mathcal{T}$ – POMDP Transitions 15
$\mathcal{R}$ – Reward Function 15
Ω – Observation Function 15
bs – Belief State 15
π – Policy 15
v – Value Function 15

CONTENTS

1 INTRODUCTION . . . 15
1.1 OVERVIEW OF RESEARCH CONTRIBUTION . . . 17
1.2 THESIS OUTLINE . . . 18

2 BACKGROUND . . . 19
2.1 NATURAL LOGIC AND INFERENCE . . . 19
2.1.1 NATURAL LOGIC . . . 19
2.1.2 NATURAL LANGUAGE INFERENCE . . . 20
2.2 BDI MODEL . . . 21
2.2.1 BDI COMPONENTS . . . 22
2.2.2 BDI REASONING CYCLE . . . 24
2.3 ENVIRONMENTS . . . 28
2.3.1 MARKOV DECISION PROCESS . . . 28
2.3.2 PARTIALLY OBSERVABLE MARKOV DECISION PROCESS . . . 29
2.3.3 TEXT-BASED ENVIRONMENTS . . . 30
2.3.4 SCIENCEWORLD . . . 31
2.4 MACHINE LEARNING . . . 32
2.4.1 ARTIFICIAL NEURAL NETWORK . . . 33
2.4.2 BACKPROPAGATION . . . 34
2.4.3 TRANSFORMER NEURAL NETWORK . . . 35
2.4.4 DEEP REINFORCEMENT RELEVANCE NETWORK . . . 38
2.4.5 SWIFTSAGE . . . 39

3 AGENT ARCHITECTURE OVERVIEW . . . 42
3.1 PROBLEM FORMULATION AND CHALLENGES . . . 42
3.2 AGENT ARCHITECTURE OVERVIEW . . . 44
3.3 BELIEF BASE . . . 45
3.4 PLAN DEFINITION . . . 46
3.4.1 NATURAL LANGUAGE PLAN LIBRARY . . . 47
3.4.2 ENTAILMENT WITH NATURAL LANGUAGE BELIEFS . . . 48
3.5 FALLBACK POLICY . . . 50
3.6 NATBDI INTERPRETER . . . 51

3.6.1 CONTROL CYCLE 51
3.6.2 EVENTS 52
3.6.3 PLAN SELECTION 54
3.6.4 FALLBACK POLICY AS INTENTION FAILURE HANDLING 55
3.7 NATBDI EXECUTION EXAMPLE 56

4 EXPERIMENTS 58
4.1 NATBDI AGENT IMPLEMENTATION 58
4.1.1 IMPLEMENTATION DETAILS 58
4.1.2 PLAN SELECTION INFERENCE 59
4.2 SCIENCEWORLD ENVIRONMENT 61
4.3 NATURAL LANGUAGE PLANS SETUP 62
4.4 FALLBACK POLICY SETUP 65
4.5 EXPERIMENT RESULTS 66
4.5.1 NATBDI RESULTS IN SCIENCEWORLD ENVIRONMENT 66
4.5.2 EFFECTS OF INCLUDING PLANS AND TRAINING EPISODES 69
4.5.3 NATURAL LANGUAGE INFERENCE ANALYSIS 70

5 DISCUSSION 74

6 RELATED WORK 77
6.1 TRADITIONAL BDI AGENTS IN NATURAL LANGUAGE ENVIRONMENTS 77
6.2 MACHINE LEARNING TECHNIQUES AND BDI AGENTS 78
6.3 LOGICAL INFERENCE ON NATURAL LANGUAGE INFORMATION 79
6.3.1 NATURAL LOGIC METHODS 80
6.3.2 LOGICAL INFERENCE IN LANGUAGE APPLYING MACHINE LEARNING 81

7 CONCLUSION 82
7.1 CONTRIBUTIONS 82
7.2 OPEN ISSUES AND LIMITATIONS 83
7.3 FUTURE WORK 84

REFERENCES 85

APPENDIX A – Natural Language Plans for ScienceWorld **93**
A.1 MELT TASK 93
A.2 FIND-NON-LIVING-THING TASK 94

1. INTRODUCTION

The increasing adoption of Artificial Intelligence (AI) algorithms in human-facing applications creates two key problems in the development of the autonomous agents that interact with humans in such applications [32, 85, 35]. First, agents must understand commands and respond to them in a human-understandable way. Second, agent developers must be able to ensure the responses generated by such agents are appropriate, regardless of the availability of training data to drive agent behaviour. Specifically, modern AI applications communicate to a certain degree with humans in their language in order to support their daily tasks [11]. Such agents process the natural language information provided by humans to make their own decisions, which can yield a natural language response or an execution of sequential steps (e.g., plans). To ensure safety and avoid unintended behaviour [35] from such applications, a scrutable mental model is essential for an autonomous agent to provide the transparency of the agent's behaviour and explain its decisions [52, 78].

Handling natural language is crucial for autonomous agents to communicate and cooperate with humans. Processing human language is a complex task for computers since information encoded by natural language is unstructured and prone to ambiguity [30]. An agent should reason over the natural language information to understand the circumstances of human problems and take an action that should be discernible to humans in order to support them. In the natural language domain, such reasoning tasks require the agent to interpret textual observations comprehending facts described in perceived information in order to decide which actions or plan should be executed [15, 28]. It requires reasoning over the context of natural language observations in order to determine whether an action or plan is applicable in a given situation.

Recent deep learning approaches for natural language processing, such as Large Language Models (LLM), achieve exceptional benchmark results when using textual information to deal with classification and generation tasks [53]. One of the key differences between such models and previous machine learning approaches is that an LLM learns word representations amenable to be reused into other NLP tasks as fine-grained input features, which speeds up the training convergence [20, 54, 39, 81]. LLMs use large amounts of natural language data capturing the semantic meaning of words grounded by the context of sentences in order to generate a textual response that satisfies the user input [72, 77]. The LLM training process includes information generated from human feedback in order to refine the language model outputs according to human preferences, which reflects how close the LLM result is to the expected answer. In order to train in such a vast amount of information, LLMs consist of neural networks with a very large number of trainable parameters, which requires significant and specialized computational resources.

Although LLMs leverage a massive volume of information contained in their training data, relying solely on such approaches to deal with more complex reasoning tasks remains a challenge. Recent work [21, 24] corroborate such argument, highlighting the limitations in problem-solving tasks that require multistep reasoning such as commonsense planning [75, 76, 69] and logical consistency [51, 24] over natural language information [79]. In fact, the LLM reasoning process is opaque [58] since such approaches rely entirely on black-box models, and hence, understanding the agent's decisions remains a difficult task. An explicit mental model representation that describes the agent behaviour is essential to understanding such reasoning limitations and dealing with them.

Traditional approaches to AI often model decision-making by borrowing terminology from folk psychology, which describes human mental attitudes to implement rational agents [9, 1, 65]. The Belief-Desire-Intention (BDI) model [6, 18] introduces a conceptual framework to implement autonomous agents composed of beliefs, desires, and intentions. We argue that incorporating plans designed by humans is instrumental for the development of more controllable autonomous agents, instructing them to avoid unintended behaviour [35]. Natural language plans allow humans to explicitly incorporate prior information which, in contrast to pure machine learning agent approaches, does not necessarily need to retrain the agent or adjust training data. Instructing the sequence of steps for an agent to perform a task does not require a training phase where the agent needs to explore or exploit states in the environment (e.g., ϵ-greedy reinforcement learning algorithms [4]). Instead, the agent can simply execute predefined plans supplied by a human.

This thesis introduces NatBDI [32], a new class of agent architecture that uses the BDI reasoning cycle with components driven by natural language processing. We leverage the advantages of modern machine learning models in natural language (e.g., pretrained language models) by using them as black-box components within the BDI model. Our contributions are threefold. First, we describe the unstructured data from observations of human activities as agent's beliefs written in natural language and generate the natural language plan library of the agent. Second, we develop an agent interpreter following the BDI mental model that uses natural language information in its reasoning back-end. This back-end uses natural language inference to find the entailment relations [16] between the belief base and the context of the plans in the plan library during plan selection. Finally, we rely on a fallback policy using black-box reinforcement learning architectures to help our agents to act autonomously in cases where the plan-selection mechanism finds no candidate (i.e., applicable) plans. Our results indicate that even adding a small amount of natural language plans when combined with the modified BDI reasoning cycle for natural language environments can dramatically increase the reasoning performance of the agent over pure machine learning agents.

1.1 Overview of Research Contribution

In this section, we list our contribution as follows:

- **Bridging the BDI Model and Real-World Problems**: We introduce a new class of BDI architecture, named NatBDI, that perceives and acts upon natural language information towards its goal. Bridging the gap between cognitive agents and real-world problems requires handling human language, as it is the primary means that humans use to represent tasks. Our approach consists of a transparent architecture whereby agents leverage natural language components based on mental attitudes, enabling humans to fully understand the agent's internal state.
- **Developing Autonomous Agents By Using Natural Language**: This thesis presents a new approach to develop BDI plans using natural language expressions. We argue that using natural language plans enables humans with no programming skills to instruct and control autonomous agents in real-world problems. Plans consist of goals, conditions, and actions encoded as imperative sentences, akin to the way humans use language to give instructions in everyday scenarios.
- **Integrating Learning and BDI as Fallback Mechanism**: As an intention failure mechanism, we introduce a novel fallback approach within the BDI architecture to instantiate actions when no plan is available. Recent works have approached decision-making problems in text-based environments by employing novel machine learning techniques. We leverage the correspondence between components from Partially Observable Markov Decision Process (POMDP) and BDI to integrate such approaches into the agent's reasoning cycle, enabling automatic plan generation based on the agent's beliefs. Inferring plans using machine learning approaches when plan library lacks candidate options reduces the dependence on human effort.

In this section, we highlight our publications made throughout the course of our Ph.D. research.The main contributions that serve as a basis of this thesis have been published in a conference as follows.

- Alexandre Yukio Ichida, Felipe Meneguzzi, and Rafael C. Cardoso. BDI Agents in Natural Language Environments [32]. Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems. International Foundation for Autonomous Agents and Multiagent Systems, 2024.

We have also published contributions related to the research topic of conversational agents, specifically modelling BDI agents to interact directly with humans using natural language. Although these publications address another research topic, which is

Task-Based Conversational Agents, their contribution is similar to this thesis as both approaches aim to apply the BDI model to problems related to human language comprehension. We have published one at a conference, and we are waiting for the response for another from a journal submission.

- Alexandre Yukio Ichida, and Felipe Meneguzzi. Modeling a Conversational Agent using BDI Framework [31]. In proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing. 2023.
- Alexandre Yukio Ichida, and Felipe Meneguzzi. A BDI Task-oriented Agent in Belief Space. In ACM Transactions on Autonomous and Adaptive Systems (TAAS)(UNDER REVIEW)

1.2 Thesis Outline

We organize this thesis as follows. In Chapter 2, we briefly describe the concepts that underlie our thesis. In Chapter 3, we detail the problem we aim to address and provide an in-depth description of the NatBDI architecture, introducing its components and an abstract implementation of its interpreter. In Chapter 4, we report the results of the experiments obtained by NatBDI in a text-based environment by analysing the effects of using different fallback policy approaches. In Chapter 5, we discuss the NatBDI architecture and its results from the text-based environment. In Chapter 6, we survey related work from the literature, categorizing them according to their research topics. Finally, we conclude this thesis in Chapter 7, reviewing our contributions and discussing open issues, limitations and avenues for future work related to our research.

2. BACKGROUND

In this section, we provide details of the basic concepts we employ in our research. We organize this chapter into multiple topics as follows: First, we detail concepts of logical inferences over natural language in Section 2.1. Second, in Section 2.2, we present the main concepts of the BDI model, which is one of the main topics of this work. Third, we describe the environment we use in this work, providing details about modelling approaches and examples of existing environments in Section 2.3. Finally, in Section 2.4, we give a brief overview of machine learning approaches that we use in our research.

2.1 Natural Logic and Inference

In this section, we provide a brief overview of methods that aims to identify logical relations in natural language information. We leverage these concepts since the NatBDI represents its mental state entirely in natural language. First, we introduce a formal system to address logical inference in natural language in Section 2.1.1. Second, we describe a method for performing natural language inferences, focusing on more informal reasoning in Section 2.1.2.

2.1.1 Natural Logic

Natural Logic is a formal system that describes logical inferences over natural language information [46]. Precisely, natural logic aims to develop a method for handling logical inferences in sentences written in natural language close to first-order logic systems. Given that most human reasoning takes place in natural language and most uses of natural language express reasoning of some sort, this model aims to create a correspondence between logical and grammatical structure [37]. Although Lakoff developed the main concepts of Natural Logic [37], Aristotle uses a similar approach to introduce syllogisms through logical arguments represented by natural language.

Syllogistic reasoning from natural logic handles simple sentences without concerning linguistic syntax but focusing on semantic consequences [46]. Formally, a hypothesis sentence s is a semantic consequence of a set of premise sentences Γ if every model of all sentences in Γ is also a model of s considering the semantic relation. That is, the semantic consequence represents an entailment relation between the sentence s with Γ induced by the linguistic semantic. The Example 2.1 shows an entailment relation between the sentence set Γ representing the premise (above the line) and the hypothesis sentence s. This example illustrates the transitivity logical property that the verb *is* con-

veys, that is, assuming that “every skunk is a mammal” and “every mammal is an animal” are true (i.e., premise), then “every skunk is an animal” is a semantic consequence of both sentences.

$$\begin{array}{l} \text{Every skunk is a mammal} \\ \text{Every mammal is an animal} \\ \hline \text{Every skunk is an animal} \end{array} \tag{2.1}$$

$$\begin{array}{l} \forall x\,(\text{skunk}(x) \rightarrow \text{mammal}(x)) \\ \forall x\,(\text{mammal}(x) \rightarrow \text{animal}(x)) \\ \hline \forall x\,(\text{skunk}(x) \rightarrow \text{animal}(x)) \end{array} \tag{2.2}$$

Notice that, as this example of syllogistic reasoning from natural logic uses sentences that are syntactically very simple, we can also represent this entailment relation in first-order logic language, which we detail in Example 2.2. Sentences from natural logic lack individual variables, comprising only instantiated terms (e.g., skunk, mammal, animal) to make logical inference, while first-order logic statements can rely on lifted variables such as x in this example. In Section 6.3, we survey related work that introduce natural logic system approaches to reason over natural logic such as monotonicity calculus [74] and semantic exclusion [41].

2.1.2 Natural Language Inference

Automated reasoning and inference are essential topics of artificial intelligence [59]. Natural language inference (NLI) is a widely-studied natural language processing task that is concerned with determining the inferential relation between a premise p and a hypothesis h [3]. In contrast to the first-order logic system, which focuses on more formal methods for performing inferences, this task addresses logical inference with more informal reasoning that encompasses linguistic aspects such as semantic knowledge, and variability of linguistic expressions [41, 14].

Recent advances in machine learning methods for natural language processing tasks introduce a wide range of approaches that leverage large text corpora to develop NLI methods [83, 20, 39]. In general, machine learning NLI approaches rely on annotated datasets [83, 42, 3] to capture the semantic aspects of sentences to infer entailment relations. That is, such approaches develop a natural logic system (recall from Section 2.1.1) that infers whether there is a semantic consequence in a sentence pair (p, h) by learning linguistic patterns from a large dataset. In the field of machine learning, NLI includes two different representations of classification tasks: two-way classification and three-way classification [40].

Two-way classification is the simplest way to represent machine learning task of NLI, which describes the task as a binary decision. The objective of this NLI task is to classify whether the hypothesis follows the premise (entailment) or does not (non-entailment). This form of classification was used in the Recognizing Textual Entailment (RTE) competition [16]. This task representation cannot detect a contradiction between *p* and *h* due to the reason that the inference of a non-entailment does not specify the relation that differs from an entailment.

Alternatively, the three-way classification deals with contradiction, creating an extra category of inferred relations. In three-way classification, the relations consist of three categories: entailment, contradiction and neutral. Given a pair of premise-hypothesis *p* and *h*, the *entailment* relation occurs when *h* can be inferred from *p* [40]. When *h* infers the negation of *p*, the pair results in a *contradiction*. Otherwise, if none of these relations can be inferred, the relation of *p* and *h* is *neutral*.

In NLI, both *p* and *h* are sentences written in natural language. The challenge of this task differs from formal deduction in classical logic due to the need to deal with informal reasoning [40]. The emphasis of the NLI is on aspects of natural language such as lexical semantic knowledge and the deal with the variability of linguistic expression. Consider the following premise *p* and hypothesis *h* as an instance of an NLI scenario [40]:

- *p*: Several airlines polled saw costs grow more than expected, even after adjusting for inflation.
- *h*: Some of the companies in the poll reported cost increases.

In the NLI context, this example is considered a valid entailment inference because any person that interprets *p* would likely accept that *h* implies in the information of *p*. Although it is a valid NLI classification, *h* is not a strict logical consequence of *p* due to the fact that *p* informs that airline companies *saw* the growth of the cost, not necessarily *reporting* the growth of the cost. This example reflects the informal reasoning of the task definition due to the ambiguity of natural language [40].

2.2 BDI Model

The Belief-Desire-Intention (BDI) model is a framework to develop autonomous rational agents bound with mental attitudes in its architecture [56]. Inspired by Bratman's philosophical work [5], the BDI framework introduces a practical reasoning approach that consists of three basic components: beliefs, desires, and intentions. BDI inherits two generic components from traditional agent architectures [82, 10], namely sensors to translate information from the external world into an internal representation,

and actuators, which translate the actions from the agent into concrete results in the environment in which the agent is embedded. The reasoning cycle of traditional BDI agents consists of two steps: deliberation and means-end reasoning [43, 18]. The deliberation step defines which state the agent should achieve, and afterwards, the means-end step defines how to achieve it. More concretely, the agent reasons over its beliefs about the environment and adopts a plan contained in the plan library to achieve intended desires. This reasoning cycle depends on the agent developer's effort to program plans by anticipating and managing all possible agent interactions over the environment. We illustrate a variant of the BDI agent architecture in Figure 2.1.

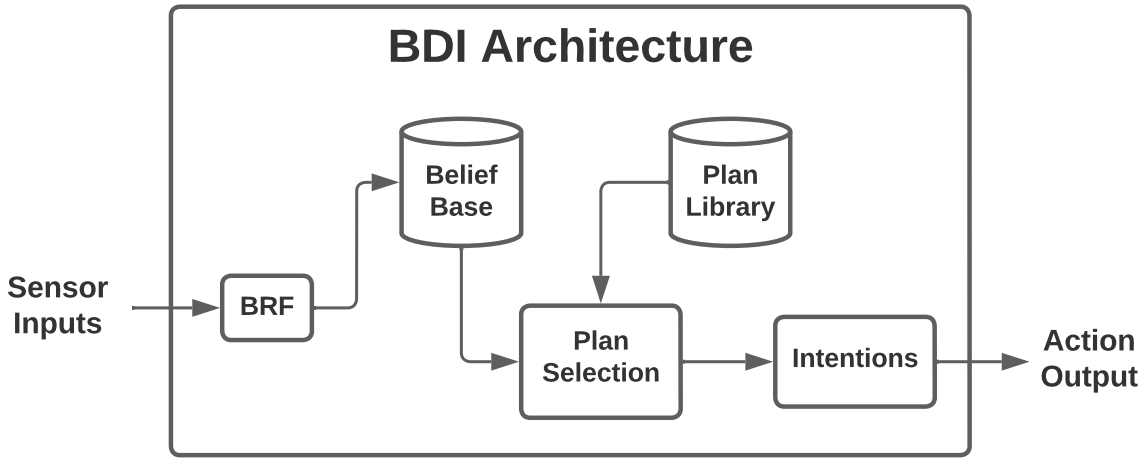


Figure 2.1 – Diagram illustrating a simple form of the BDI agent architecture and its components.

In this section, we further detail the BDI model as follows: Section 2.2.1 provides a brief description of the main components in the BDI agent architecture and details how the BDI model handles plans. In Section 2.2.2, we describe an abstract implementation of a BDI interpreter to detail how the BDI reasoning cycle employs its components to handle tasks.

2.2.1 BDI Components

In this Section, we detail further components and definitions that comprise the BDI model using the AgentSpeak language [55], which is a well-known language to program BDI agents. Besides belief, desire and intention, as mentioned before, we describe other structures involved in the agent's reasoning cycle, such as plan library and event queue. According to the definition formulated in Meneguzzi and de Silva [43] work, we formalize the BDI agent components as follows:

Definition 2.2.1 (BDI Agent)**.** A BDI agent is a tuple $\langle Ag, \mathcal{E}, \mathcal{B}, \mathcal{PL}, \mathcal{I} \rangle$, where Ag is the agent identifier; $\mathcal{E}$ is the queue of events; $\mathcal{B}$ is the belief base; $\mathcal{PL}$—the plan library—is a set of plan rules; and $\mathcal{I}$—the intention structure—is a set of intentions.

Belief represents information that describes what the agent understands about the world based on its perceptions of the environment. In traditional BDI agents, beliefs encode information into grounded predicates represented by symbols defined by the agent developer. For instance, the ground predicate $in(glassjar, cupboard)$ represents a belief that describes the perception of a glass jar inside a cupboard. During the agent execution, the agent observes events from the environment and might include new beliefs or delete the existing ones in a structure named Belief Base, which stores all agent perceived beliefs. In order to maintain consistency in beliefs, the BDI architecture includes a function to control changes in the belief base named Belief Revision Function (BRF).

Definition 2.2.2 (Beliefs and Belief Base)**.** A belief φ is a grounded predicate that represents the agent's perceptions. The belief base $\mathcal{B}$ is a structure that stores a finite set of beliefs.

Desires represent states of affairs that the agent aims to achieve [56]. The agent can have different goals during its execution, and each one can trigger different plans to pursue them. In BDI languages such as AgentSpeak [55], there are two types of goals: achievement goals and test goals. Achievement goals consist in a true belief that represents the state the agent wants to achieve. Test goals are grounded predicates representing a formula the agent wants to verify whether it is true according to its belief base.

In BDI, the reasoning cycle relies on an event-driven approach to manipulate the belief base state and the goal/plan adoption. This approach stores all events perceived during the reasoning process into a structure named *event queue*. The role of events in belief base maintenance is to control either inclusion of new beliefs (i.e., *belief addition* event), or deletion of existing ones (i.e, *belief deletion* event). Regarding plan adoption, the reasoning cycle uses triggering events to decide which plan should be executed based on a goal addition or deletion. Formally, we follow Meneguzzi and De Silva [43] definition to describe events structure as below:

Definition 2.2.3 (Events)**.** Given a predicate ϕ, an event e is either:

1. a belief addition expressed as $+\varphi$, with φ representing a belief added in belief base $\mathcal{B}$;

2. a belief deletion expressed as $-\varphi$, with φ representing a belief removed from belief base $\mathcal{B}$;

3. a goal addition expressed as $+!\varphi$, with φ representing a new goal for the agent to achieve;

4. a goal deletion expressed as $-!\varphi$, with φ representing a goal dropped by the agent;
5. a test goal addition expressed as $+?\varphi$, with $?\varphi$ representing a test goal is posted to the agent; or
6. a test goal deletion expressed as $+?\varphi$, with $?\varphi$ representing a test goal is dropped by the agent.

Implementations of BDI agents encode their behaviour as plan-rules consisting in two parts: the plan header, which describes when the plan can be executed; and the plan body, which contains a sequence of steps for the agent to perform. The plan header informs the triggering event of the plan in addition to the plan context, which describes conditions to consider it as a candidate option. The plan context comprises in grounded predicates representing a first-order formula over the agent's belief base, that is, to be considered as a candidate option, the belief base should entail the plan context.

Definition 2.2.4 (Plan rules and plan library)**.** A plan rule $\mathcal{P}$ is a tuple $\langle t, \mathcal{C}, bd \rangle$, where t is the triggering event to call $\mathcal{P}$; $\mathcal{C}$ is the context condition encoded in a first-order formula indicates whether a plan can be adopted as a candidate given the current belief base; and bd is the plan body consisting in the steps to achieve the goal. A plan library $\mathcal{PL}$ is a set of plan rules $\{\mathcal{P}_1, ..., \mathcal{P}_n\}$, which the agent queries to find candidate plans.

The intention component is a structure that consists of a set of instantiated plans adopted by the agent in order to achieve a subset of its desires. This component is a stack that contains either actions, which the agent can execute directly in the environment, or an event to trigger another plan. When an intention includes a goal addition event, the agent executes another plan and decomposes its steps back onto the previous intention stack, which allows developing plans in a hierarchical approach [61]. Agent developers implement plans to instruct the agent how to achieve a goal by specifying a sequence of steps along with the context of their applicability, storing them in a component named Plan Library.

Definition 2.2.5 (Intention structure)**.** An intention structure $\mathcal{I}$ is a set of intentions, and each intention is a tuple $\langle \sigma, st \rangle$ containing a substitution σ, and a set of steps st in a stack indicating the agent how to accomplish the goal.

2.2.2 BDI Reasoning Cycle

In this section, we describe a basic BDI interpreter and how the components described in Section 2.2.1 operate within the BDI reasoning cycle. Initially, the BDI agent perceives events from environment changes or goal definitions and includes them in its

event queue. In cases where events indicate changes in the environment, the agent updates its belief base by checking which beliefs should be included (i.e., belief-addition event) or removed (i.e., belief-deletion event). In order to start actuating over the environment, the agent receives its goal through events in order to select a plan to achieve it. Given a candidate plan, the agent executes intentions from its plan body, which can generate other events or lead to action execution. We follow the Meneguzzi and De Silva [43] work to specify an abstract implementation of a BDI interpreter in Algorithm 2.1.

```
1: procedure bdiInterpreter(⟨Ag, ℰ, ℬ, 𝒫ℒ, ℐ⟩)
2:     while True do
3:         ℰ ← updateEvents(ℰ)
4:         ℬ ← updateBeliefs(ℬ, ℰ)
5:         ℰ, ℐ ← selectPlans(ℰ, ℬ, 𝒫ℒ, ℐ)
6:         ℰ, ℐ ← executeIntentions(ℰ, ℐ)
```

Algorithm 2.1 – Abstract implementation of a BDI interpreter [43].

The interpreter receives the BDI agent information following Definition 2.2.1 to operate over its components. The function to update events shown in Algorithm 2.2 gathers the perceived event, pushing it to the event queue $\mathcal{E}$ and returns its updated version ($\mathcal{E}'$). It assumes the existence of a function perceiveEvents that provides perception encapsulated as events in some manner. The updateBeliefs function scans all events from $\mathcal{E}$ and, for each $e \in \mathcal{E}$, it verifies whether e represents a belief addition or belief deletion, which we describe in Algorithm 2.3. In updateBeliefs function, the BDI interpreter either includes the belief predicate (φ) in the belief base (Line 5 of Algorithm 2.3) when a belief addition event is perceived or removes it (Line 7 of Algorithm 2.3) when a belief deletion event is perceived.

```
1: function updateEvents(ℰ)
2:     e ← perceiveEvents
3:     ℰ′ ← push(e, ℰ)
```

Algorithm 2.2 – Function that updates the event queue given perceptions obtained from environment [43].

The selectPlans function (Algorithm 2.5) dequeues the next event from the event queue in order to find options in the plan library that satisfy the invocation condition. Given the plan $\mathcal{P}_i$ containing a triggering event t, and the plan context $\mathcal{C}$, if an event e unifies with t via substitution σ and the context $\mathcal{C}$ is entailed by the belief base $\mathcal{B}$, then $\mathcal{P}_i$ is a candidate option (Line 6 of Algorithm 2.4) for achieving the agent's goal [43]. We detail these operations in function selectOptions, illustrated in Algorithm 2.4, which details how the BDI interpreter acquires a list of plan candidates along with their substitution σ_{opt}.

```
1: function updateBeliefs(ℰ, ℬ)
2:     ℬ′ ← ℬ
3:     for e ∈ ℰ do
4:         if e = +φ then
5:             ℬ′ ← ℬ′ ∪ {φ}
6:         else if e = −φ then
7:             ℬ′ ← ℬ′ − {φ}
       return ℬ′
```

Algorithm 2.3 – Function to update belief base according to the belief addition/deletion events perceived in event queue [43].

In the BDI model, options are plans that satisfy the conditions set in their respective plan rules to become intentions, that is, plans that have its plan context entailed by the agent's belief base.With an option queried from the plan library, the interpreter includes it into the intention structure. In cases where the event represents a test goal, if the belief base entails the belief being tested, the interpreter adds a new intention with the unifier of that belief test. When there is no option available in the plan library (i.e., no plans triggered by the event e), then the interpreter indicates a failure in the intention that generated the event by calling the intentionFailure function. We assume there is a function that handles the failure since this is an abstract implementation of a BDI interpreter and there is a wide option for handling BDI failures [43, 1].

```
1: function selectOptions(e, ℬ, 𝒫ℒ)
2:     Opt ← ∅
3:     for plan ⟨t, 𝒞, bd⟩ ∈ 𝒫ℒ do
4:         for σ, σ′ such that unify(e, t, σ) ∧ (ℬ ⊨ 𝒞(σ, σ′)) do
5:             σ_opt ← σ · σ′
6:             Opt ← Opt ∪ {⟨e, 𝒞, bd, σ_opt⟩}
       return ℬ′
```

Algorithm 2.4 – Function that queries the plan library and returns the candidate plan [43].

Algorithm 2.6 describes the addIntention function used in selectPlans to include a new intention to the intention structure. Initially, it instantiates a new intention encapsulating the event e, the plan body bd, and the substitution σ_{opt} obtained from the unify function of event e with the plan invocation condition t mentioned before. After generating the new intention, the interpreter appends it into the original intention structure $\mathcal{I}$ and returns the updated intention structure $\mathcal{I}'$. Assuming the intention that generated the event is accessible through the event e (int_e, Line 3), the createIntention function (Algorithm 2.7) generates a new intention ($\langle \sigma'_{int}, st' \rangle$) by using a new substitution σ'_{int} and a new intention stack st', which is composed of the st of int_e jointly with the plan body with σ'_{int} applied (Line 5).

```
1:  function selectPlans(e, B, PL, I)
2:      I' ← ∅
3:      E', e ← pop(E)
4:      Opt ← selectOptions(e, B, PL)
5:      if Opt ≠ ∅ then
6:          Pick one option ⟨t, C, bd, σ_opt⟩ ∈ Opt
7:          I' ← addIntention(e, bd, σ_opt, I)
8:      else if (e = +?φ) ∧ (B ⊨ φσ_b for some σ_b) then
9:          I' ← addIntention(e, ∅, σ_b, I)
10:     else
11:         E', I' ← intentionFailure(int_e, I, E')
        return E', I'
```

Algorithm 2.5 – Function to select plans and fill the intention structure [43].

```
1:  function addIntention(e, bd, σ_opt, I)
2:      I' ← ∅
3:      int_e is the intention that generates e
4:      ⟨σ, st⟩ = createIntention(e, bd, σ_opt)
5:      if e  is in either of the forms +!φ, −!φ, +?φ, or −?φ then
6:          I' ← (I − {int_e}) ∪ {⟨σ, st⟩}
7:      else
8:          I' ← I ∪ {⟨σ, st⟩}
        return I'
```

Algorithm 2.6 – Function to add new intentions in the agent intention structure [43].

```
1:  function createIntention(e, bd, σ)
2:      if e  is in either of the forms +!φ, −!φ, +?φ, or −?φ then
3:          int_e ← ⟨σ_int, st⟩ int_e is the intention that generates e.
4:          σ'_int ← σ_int · σ
5:          st' ← push((bdσ'_int), stσ'_int)
            return ⟨σ'_int, st'⟩
6:      else
        return ⟨σ, bdσ⟩
```

Algorithm 2.7 – Function to instantiate intentions given a new intention stack with all substitutions applied [43].

After the plan selection phase, the interpreter executes intentions from the intention structure as detailed in Algorithm 2.8. The function executeIntention retrieves the next step contained in the intention stack and verifies whether it is an action or an event. The interpreter executes actions immediately in the environment (Line 6), while in cases where intention generate events, the interpreter adds them into the event queue to proceed the next reasoning cycle (Lines 12, 14). When the executed action is the last step from intention stack (Line 7), that is, the remaining intention stack is empty, then the intention is removed and the function execution terminates (Line 9). We abstracted the

executeAction function since it depends on how the environment in which the interpreter is running is implemented.

```
function executeIntention(𝓘, 𝓔, 𝓑)
    𝓔′, 𝓘′ ← ∅
    int ← topmost intention from 𝓘
    int′, s ← pop([s_1, ..., s_n])
    if s is an action then
        executeAction(s)
        if int = ⟨σ, []⟩ then
            𝓘′ ← 𝓘 − {int}
            return 𝓔′, 𝓘′
    else if s is a subgoal g then
        𝓔′ ← push(+gσ, 𝓔)
    else if s is a belief event then
        𝓔′ ← push(sσ, 𝓔)
    𝓘′ ← (𝓘 − {int}) ∪ {int′} //removes the old intention and includes the new one.
    return 𝓔′, 𝓘′
```

Algorithm 2.8 – Function to instantiate intentions given a new intention stack with all substitutions applied [43].

2.3 Environments

In this section, we detail the environment specifications that we handle in our work. Sections 2.3.1 and 2.3.2 describe approaches for modelling sequential decision-making, which underpin the implementation of the natural language environments used in this thesis. Section 2.3.3 introduces concepts of text-based environment detailing how states and actions are represented. Section 2.3.4 describes a text-based environment instance named ScienceWorld, which we use in our research experiments.

2.3.1 Markov Decision Process

A Markov Decision Process (MDP) is a modelling approach to address sequential decision problems that require the agent to perform a sequence of actions to achieve its goal [71, 59]. In an MDP, the agent interacts with the environment by transitioning between fully observable states and receiving a feedback signal indicating how good the transition is from the previous state to the current one. Formally, we describe an MDP as follows:

Definition 2.3.1 (MDP)**.** An MDP comprises a 5-tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma \rangle$, where $\mathcal{S}$ represents a set of the environment states; $\mathcal{A}$ a set of actions; $\mathcal{T}$ a transition function; $\mathcal{R}$ the reward function; and γ the discount rate, $0 \leq \gamma \leq 1$.

The goal of MDPs is to acquire a decision-making function π, named policy, that maps a state $s \in stateset$ to an action $a \in \mathcal{A}$. That is, an MDP aims to find a policy π that returns an action a that maximizes the expected discounted return G_t, which incorporates the discount rate γ from Definition 2.3.1. The discount rate is the present value of future rewards, which measures how myopic or far-sighted G_t is. For example, if γ is 0, the G_t value considers only the immediate reward (i.e., $\mathcal{R}$), whereas values closer of 1 consider a more long-term reward. We describe G_t in the equation below:

$$G_t = \sum_{k}^{\infty} \gamma^k \mathcal{R}_{t+k+1} \tag{2.3}$$

2.3.2 Partially Observable Markov Decision Process

Partially Observable Markov Decision Process (POMDP) is a formal model for an agent's decision-making in cases where it is not possible to access the complete state information [59]. That is, the agent makes decisions under uncertainty by relying on observations that provide partial information about the current state. In this case, the agent needs to observe its surroundings to gather more information about the environment. To measure the uncertainty of observations, the agent uses a sensor model to formulate a function that returns the probability of perceiving an observation given a specific state. Formally, we describe a POMDP model in the following definition.

Definition 2.3.2 (POMDP)**.** A POMDP model tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{T}, \Omega, \mathcal{R} \rangle$, where $\mathcal{S}$ represents a set of the environment states, $\mathcal{A}$ a set of actions, $\mathcal{O}$ a set of observations, $\mathcal{T}$ a transition function, Ω the observation probability function, and $\mathcal{R}$ the reward function.

However, since states are not fully observable, a POMDP agent should compute a probability distribution over all actual states to generate the belief state [63]. A valid option to initialize the probability distribution of the belief state could be applying a discrete uniform distribution using $|\mathcal{S}|$. Given an observation $o \in \mathcal{O}$ perceived and actions performed by the agent, we compute the belief state $bs(s')$ of a state s' as follows:

$$bs(s') = \alpha \Omega(o|s') \sum_{s} \mathcal{T}(s'|s, a) bs(s) \tag{2.4}$$

Equation 2.4 formulates a recursive Bayesian update to computing $bs(s')$ to predict the s' probability. It multiplies the probability of the new observation o given the state

s' (i.e., $\Omega(o|\ s')$) with the sum of all belief state of states $s \in \mathcal{S}$ considering the transition to s' from $s$by taking action a (i.e., $\mathcal{T}(s' \mid s, a)$). The α term is a constant that normalizes the probability distribution to makes the sum of belief states to 1.

In contrast with fully observable Markov Decision Process (MDP), which uses a policy to map states to actions; in POMDP, we cannot rely on this definition since the agent does not have certainty of the current state. Instead, we can use the agent's belief state to predict the next action, that is, using a policy as a function to map the current belief state to an action [59]. In this case, the policy of POMDP agents does not depend on the actual state to predict its next action but the current belief state.

2.3.3 Text-Based Environments

Text-based environments are simulated environments that represent the world information into natural language descriptions. Interactive Fiction games are an instance of text-based environment where the player should interpret textual observations of the simulated world and take text actions to progress through the game story [28, 15]. Such an environment aims to evaluate the competencies of players in terms of natural language understanding, common sense reasoning, and deduction to make decisions to achieve the game objective. Regarding developing autonomous agents to deal with real-world problems, text-based environments provide a challenging test bed for designing agents capable of understanding natural language and interacting effectively with humans.

Implementations of text-based environments [28, 80, 15] model it as a POMDPs model assuming the sentences in the perceived observation do not express the entire game world, that is, the agent cannot assume full observability in such environments. For example, textual observations convey information about what the agent is seeing in the current location, which requires the agent to explore the world in order to know what other environment locations have. We leverage Definition 2.3.2 to specify each component in text-based environment as follows: $\mathcal{O}$ corresponds to sentences describing the performed action effects in the environment, $\mathcal{A}$ to textual actions encoded in imperative sentences (e.g., go outside), $\mathcal{S}$ to latent states encoded in a textual description informing location details, object seen, and objects in inventory.

When an agent executes an action, the environment sends back an updated version of the state, considering the effects and changes that have occurred. Specifically, for each agent's performed action, the environment responds with two textual information: a sentence informing effects caused by the performed action (i.e., observation), and a paragraph describing the situation of the current location after performing the action. Besides the texts informing the current environment condition, the environment sends a

reward considering an unknown reward function $\mathcal{R}$, along with a boolean signal informing whether the agent could achieve the environment objective.

2.3.4 ScienceWorld

Science World is a textual environment that simulates engines for thermodynamics, electrical circuits, matter and chemistry reactions, and biological processes [80]. This environment aims to create a benchmark to evaluate scientific reasoning abilities of autonomous agents at the level of the standard elementary school science curriculum. The environment provides 30 benchmark tasks across 10 topics covering understanding changes in the state of matter, building electrical circuits, and the pollination process. Similar to Jericho framework [28], the Science World environment provides a set of valid actions given a specific state and the object model to support the development of autonomous agents.

Science World map contains 10 distinct locations around a house (kitchen, bathroom, workshop, art studio, greenhouse, outside, etc.) and 195 specific types of objects. Each location contains a set of random objects contextually relevant to the task objective. The Science World object model includes 23 animals, 11 plants, 25 substances, 10 canonical liquid containers (like tin cups or glass jars), 13 electrical components (such as light bulbs, motors, wires, and generators), 16 devices (including a stove, thermometer, and stopwatch), and 15 common pieces of furniture (e.g., tables, chairs, etc.) [80].

Such complex settings aim to evaluate the agent's reasoning about location transition and object interaction and test understanding capabilities about combining distinct objects considering their state of matter. For example, in order to boil the water, the agent should walk around the map to find a metal pot, search for water, and use a stove. In this test case, the agent should leverage prior knowledge about the object state not to choose a wooden cup instead of the metallic one since it will have contact with fire. Furthermore, the agent should use the thermometer object to be aware of the water state since waiting too much can turn the water into vapour.

Each task in the Science World has a description of the main objective written in natural language with high-level instructions for each sub-goal. The state consists of a set of sentences that describes the objects available in the current room and which rooms the agent can go through according to the Science World map. For each action execution, the environment gives a score as a reward signal in order to apply reinforcement learning methods following the implementation of POMDP text-based environments, as we detail in Section 2.3.3). Throughout the agent interaction in the environment, the task score

increases as the agent approaches the task objective. Figure 2.2 illustrates an example of tasks provided by the Science World web application[1].

Science World (Text Simulation)

Task: Your task is to boil water. For compounds without a boiling point, combusting the substance is also acceptable. First, focus on the substance. Then, take actions that will cause it to change its state of matter.
Variation: 1

Move 0

```
This room is called the art studio. In it, you see:
        a large cupboard. The large cupboard door is closed.
        a wood cup (containing blue paint)
        a wood cup (containing red paint)
        the agent
        a substance called air
        a wood cup (containing yellow paint)
        a table. On the table is: a bowl (containing nothing).
You also see:
        A door to the hallway (that is closed)
```

Task Score: 0 (isCompleted: False)

User Input:
> *open door to the hallway*

Move 1

```
The door is now open.
```

Task Score: 0 (isCompleted: False)

Figure 2.2 – Example of Science World task with the state encoded in natural language and the action *open door to the hallway* selected.

2.4 Machine Learning

Machine learning is a field of study that focuses on computational approaches to handling tasks without being explicitly programmed [47]. Such approaches consist of algorithms that leverage prior information about the problem domain and identify patterns to generate a result. Concretely, machine learning approaches comprise a training phase to obtain an optimized model from data, and an inference step to predict the output using the trained model. Mitchell [44] formally details machine learning as follows: "A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P if its performance at tasks in T, as measured by P, improves with experience E.". We divide machine learning approaches into the following categories according to how they handle the feedback in the training step: *supervised learning*, *unsupervised learning*, and *reinforcement learning* [47].

[1] https://github.com/allenai/ScienceWorld. Last accessed on March 10, 2025

Supervised learning approaches aim to obtain a model using annotated data that includes labels as feedback during training. Annotated data consists of a dataset containing a set of input properties, named features, along with a label representing the expected output [59]. Formally, given a function f that represents a model with parameters w, the input x, and its corresponding label y, such approaches search w values that minimize the difference between $f(x)$ and y. For instance, given a market forecasting problem, we can apply a supervised learning approach to predict future sales by identifying patterns in past sales data. The dataset includes input features such as past dates annotated with the corresponding sales quantities.

In contrast, unsupervised learning approaches focus on training a model that does not rely on an annotated dataset. The training dataset for unsupervised learning approaches does not contain any labels to be used as feedback, making it easier to obtain since it does not require effort in data annotation [59]. Clustering tasks are common applications that unsupervised learning approaches can deal with when there is no prior information about each cluster. Formally, in such tasks, an unsupervised approach obtains a function f that infers the clusters of a unsupervised dataset based on feature similarity, that is, f assigns similar inputs to the same cluster.

Reinforcement learning includes approaches that develop a decision-making model to guide the agent's actions for performing tasks in an environment [71]. In this case, training information consists of transitions that the agent obtains by exploring the environment, and a reward signal representing feedback that the agent receives by performing actions in such transitions [59]. In reinforcement learning, the transition states represent the model's input whereas the action is the output, indicating the action that the agent should perform. Obtaining an optimal policy in MDPs is a task that reinforcement learning approaches aim to address. That is, the objective of the reinforcement learning training process is to acquire a policy that maximizes the expected total reward.

2.4.1 Artificial Neural Network

Artificial neural networks are computational models for machine learning inspired by the neural networks of the biological brain. The neural network learning approach structures the model as a directed acyclic graph, where nodes represent values and edges represent the model parameters. Organized into multiple interconnected layers, the neural networks model executes intermediate computation along the graph's direction, which are called activation functions. Concretely, a neural network implementation represents nodes and edges as numerical matrices, where the number of nodes determines the matrix dimensionality in each network layer, which we illustrate in Figure 2.3.

The input layer represents the neural network's input features, and the output layer represents the neural network results. A neural network has one or more hidden layers that perform intermediate computation, representing the neural network's depth. Each hidden layer may include an additional node called bias, which represents a fixed value that the neural network propagates to the next layer.

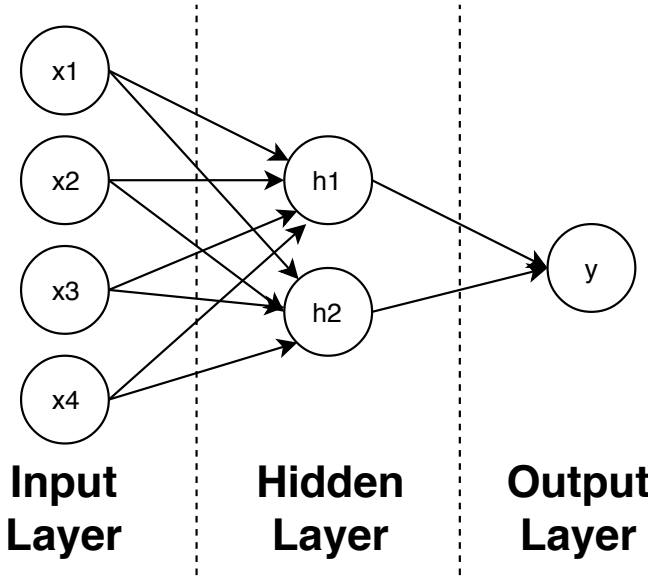


Figure 2.3 – Graph representing a neural network model and its layers.

2.4.2 Backpropagation

Backpropagation is an algorithm to train neural networks by updating their weights through error minimization between the predicted and the expected values [44]. Recall from Section 2.4.1 that neural network may have multiple layers, making it challenging to determine the weight update values. The Backpropagation algorithm measures the error to adjust weights by employing the *Chain Rule* of calculus to compute the derivatives of activation functions of the neural network layers [26]. It assumes that neural network correspond to a composition of multiple differentiable functions and, hence, each layer's activation function should be differentiable in order to update the weights. This algorithm is divided into two steps: forward propagation and backward propagation.

In forward propagation, the algorithm executes a prediction over input values computing the activation functions of all the network layers [26]. The input of the activation function is the result of the matrix multiplication between the layer input values and its weights, including the sum with the bias value. Equations 2.5 and 2.6 show the operations processed in a neural network layer, where the variable j represents the neuron index of the layer l, the variable k represents the neuron index of the previous layer $(l-1)$ and the operation a_j^l represents the activation function σ.

$$z_j^l = \sum_k w_{j,k}^l \cdot a_k^{l-1} + b \tag{2.5}$$

$$a_j^l = \sigma(z_j^l) \tag{2.6}$$

After forward propagation reaches the output layer predicting the neural network output, then the backward propagation is executed. This step of the algorithm relies on the Chain Rule to propagate the error measured from the output layer to all of the neural network weights. Equation 2.7 shows the Chain Rule applied in the backward propagation with variable J representing the cost function and variable δ^L representing the error to be back propagated from the output layer L.

$$\begin{aligned} \delta^L &= \frac{\partial J}{\partial z^L} \\ &= \frac{\partial J}{\partial a^L} \cdot \frac{\partial a^L}{\partial z^L} \end{aligned} \tag{2.7}$$

2.4.3 Transformer Neural Network

Transformer is a type of neural network architecture that processes sequences based solely on attention mechanisms instead of using recurrent connections in the network. Vaswani *et al.* [77] developed this architecture to deal with the machine translation task, achieving state-of-the-art performance on natural language processing tasks.This architecture uses an Encoder-Decoder approach based on other machine translation neural networks such as Sequence to Sequence learning [70]. The encoder and decoder of Transformer comprise blocks that contain two types of neural network layers: a self-attention layer and a feed-forward layer.

The self-attention mechanism learns the internal relations between elements of an input sequence, which is a significant part of this architecture. These relations allow a Transformer to learn semantic representations that carry information about all elements in the same sequence and learning long term dependencies as well. Given that these neural networks learn term dependencies that rely only on a self-attention layer, a recurrent neural network with memory gates is not necessary in this architecture.

On the other hand, since a recurrent mechanism is absent [13], the input of Transformer architecture does not have information about the order of elements. To deal with this issue, the authors of the Transformer architecture introduce a representation that informs the element position in its representation, known as Positional Embedding. The input of the Transformer architecture is the sum of input embedding with its Positional Em-

bedding. Vaswani *et al.* [77] use a sinusoidal function to represent Positional Embeddings of each position.

A self-attention layer comprises three matrices: a query matrix, a key matrix, and a value matrix [77]. Each element of a sequence is associated with a query matrix that maps an output given a pair of key/value matrix, which projects the magnitude of relations with other elements. Although encoder and decoder uses self-attention mechanism similarly, the decoder part of Transformers applies a mask to remove later element values to avoid computing the attention score to subsequent positions of a single word. Equation 2.8 shows the function of the self-attention mechanism that results in a representation Z given a query matrix Q, a key matrix K, a value matrix V and a normalization term d_k, which represents the dimensionality of matrix K. Figure 2.4 describes an example detailing each operation of the self-attention mechanism, which produces the attention weights to focus on relevant words and then multiply with its values.

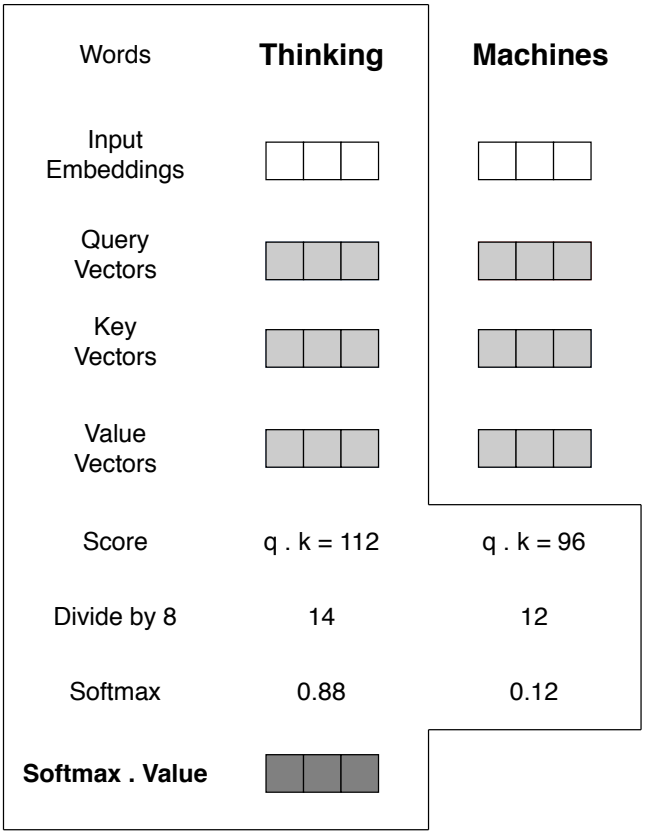


Figure 2.4 – Diagram of operations executed by self-attention mechanism, generating the representation of the word “thinking” given the input sentence “Thinking Machine”. The numbers are hypothetical.

$$Z = softmax(\frac{Q.K}{\sqrt{d_k}}).V \tag{2.8}$$

Besides the computation of relations between elements in a sequence, this architecture introduces the approach of computing multiple representations for each sequence

element, known as Multi-Headed attention [77]. The objective of Multi-Headed attention is to generate representation subspaces of an element to learn different aspects of an individual element, in which each subspace represents an attention mechanism head. Each head contains a distinct query, key and values matrices randomly initialized and computes the self-attention mechanism as well as explained previously. After computing each head, the neural network concatenates each head output into a single matrix and then transforms it into a matrix with the same shape of input size using a fully-connected layer. Figure 2.5 shows how each subspace of multi-head attention computes the output representation.

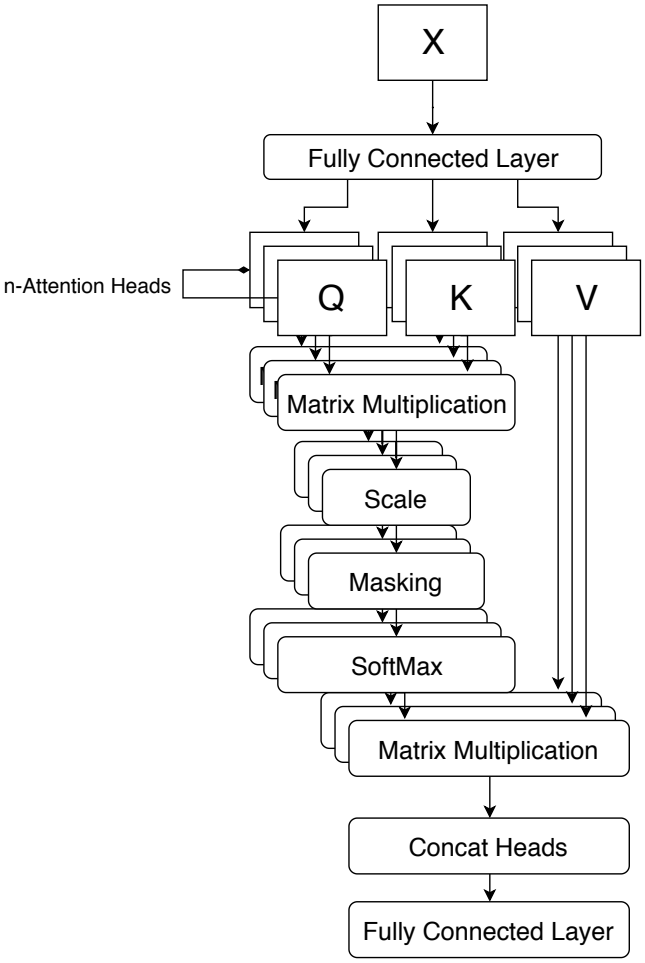


Figure 2.5 – Diagram of self-attention model using Multi-Head approach using 3 heads.

After computing the self-attention layer, the Transformer block uses a feed-forward neural network to process each hidden state of the sequence separately and identically. The transformer block uses an extra layer applying the Batch Normalization method [34] on the output of each block layer.

2.4.4 Deep Reinforcement Relevance Network

Deep Reinforcement Relevance Network (DRRN) is a reinforcement learning approach to make decisions in an environment written in natural language [29]. Given an observation perceived from a text-based environment, DRRN selects the most relevant action from a candidate action list by using a relevance function. Such relevance function receives both state and action vector representations (i.e., embeddings), and uses their inner product as a numerical output to indicate how relevant is each action with respect to the input state. DRRN derives the relevance function by leveraging the Q-learning algorithm, which estimates a Q-function by processing each state-action pair seen during the training step and assigning its Q-value as the numerical representation of relevance.

To obtain both action and state representations, DRRN employs a Siamese neural network to learn distinct finite-dimensional embedding spaces for state and action texts [29]. Precisely, the DRRN neural network consists of two Gated Recurrent Units (GRU) [13] networks to encode each candidate action and the state into embedding vectors, along with a numerical relevance function [29] at the top of the network that computes the inner product of both representations. We formally describe the DRRN neural network in Equation 2.9, where h_s and h_a represent vector representations of state and action respectively, and $Q(s, a)$ is the Q-value for the input state-action pair (s, a) consisting of the inner product of h_s and h_a.

$$\begin{aligned} h_a &= GRU(a) \\ h_s &= GRU(s) \\ Q(s, a) &= h_a \cdot h_a \end{aligned} \tag{2.9}$$

The DRRN training process consists of exploring the environment and storing the transitions into a Prioritized Replay Buffer [62]. This process samples a mini-batch of transitions from the replay buffer consisting of a tuple(s, a, r, s', a'), where s' and a' represent the next state-action pair, and r is the reward associated with performing a. In order to optimize the network weights, the DRRN training process minimizes the temporal difference δ to employ the backpropagation algorithm for network optimization.

$$\begin{aligned} s, a, r, s', a' &\sim \mathcal{D} \\ \delta = r + \gamma * \max_a Q(s', a') &- Q(s, a) \end{aligned} \tag{2.10}$$

2.4.5 SwiftSage

SwiftSage is a framework to develop agents to perform tasks over text-based environment using large language models [38]. Inspired by the dual process theory [36], the framework consist of two main modules: First, a module to emulate the fast and intuitive thinking, named Swift. Second, a module to represent the slow and deliberate thought process, named Sage. The framework introduces a heuristic to integrate both modules to create a problem-solving process by perceiving natural language information and generating actions to solve tasks.

The swift module consists of a smaller large language model that maps short-term information perceived from the environment, such as action history, observations, and visited locations, to a natural language action. Lin et al. [38] applies behaviour cloning to train a T5-large model [54] by mimicking oracle trajectories, assuming there is a sequence of steps annotated for each task seen in the training step. This module relies on an input prompt format in the following format:

- "Task: D; Time t; Score: S; Action history: [$A_{t-1}(+R_{t-1)})$ -> $\mathcal{O}_{t-i}, \ldots, A_{t-i}(+R_{t-i)})$ -> $\mathcal{O}_{t-i}$]; Current room: L; Inventory: I; Visited rooms: E", where:
  - D is the task description that the environment provides to the agent.
  - E is the list of visited rooms.
  - t is the current turn.
  - A is a set of actions performed previously.
  - R is the reward received.
  - L is a text describing what the agent perceives in the current room.
  - I is a text describing the agent's inventory.
  - O is the observation perceived after performing the action $a \in A$.

Listing 2.1 describes a concrete example of an input prompt for finetuning the Swift Module.

Listing 2.1 – Swift input prompt example to represent the input state during training step containing T5 special tokens in its content (e.g., <extra_id_1>, <s>).

```
Task: Your task is to melt ice cream.; Time: 14; Score: 35; Action history:
<extra_id_0> Action 5 (+5): open fridge —> You opened fridge. In it, you see
an ice cream... <extra_id_9> Action 14 (+0): move metal pot to stove —>
You move the metal pot to the stove. </s> Current environment: This room is
kitchen. You see: a fridge (closed), a sink, an oven (closed, turned off).
Inventory: an orange </s> Visited: workshop, hallway, kitchen </s>
```

The Swift module training process leverages existing trajectories that comprise in a smaller sequence of steps that ensure the completion of the task. Considering that the training process uses behaviour cloning from observations [73] by using such trajectories, the trained model that grounds the Swift module often fails to generalize to situations that significantly diverge from the ones seen [25]. This leads the model to generate meaningless actions in out-of-distribution states, which generates repeated errors when the predicted action generates exceptions from the environment (e.g., trying to activate a broken stove).

In order to address this limitation, Lin et al. [38] include a component to emulate slow thinking based on a Large Language Model (LLM), named Sage. This approach consists of creating a textual prompt asking a set of questions in order to obtain the plan to deal with the task, using ChatGPT-4 as the LLM implementation [7]. Specifically, the Sage module process consists of two staging prompts: the planning stage and the grounding stage. The planning process consists of generating a prompt that includes prior information perceived from the environment, which is similar to the Swift prompt format, along with the following questions:

- Q1 (locate objects): “To complete the task, which objects do I need to collect? Please list them and their possible locations one by one.”
- Q2 (track objects): “Are there any objects that have not been collected yet?”
- Q3 (plan subgoals): “To complete the task most efficiently, what are the important subgoals to achieve? Please list the subgoals one by one.”
- Q4 (track progress): “Considering these subgoals, what have I already completed? And which subgoal should I focus on right now?”
- Q5 (handle exceptions): “Have I made any mistakes that might prevent me from efficiently completing the next subgoal? If any, how should I fix them?”

Based on the answers that the LLM generates for all the questions above, the grounding stage aims to convert the LLM’s text output from the planning stage into executable actions (i.e., valid environment actions). In this stage, the Sage module generates a second prompt requesting to the same LLM the conversion of such answers into valid actions for the environment and stores the generated actions in a buffer, which the agent should execute. The grounding stage prompt comprises in instructions informing which actions are valid for the environment using a formal style to represent them. Listing 2.2 describes a set of action examples with a text for additional information that the Sage module includes in its prompt to the LLM.

Listing 2.2 – Action instructions that the Sage module incorporates into its prompt.

```
TELEPORT(room) : directly go to a room such as TELEPORT(kitchen)
```

```
PICK(object) : pick up an object and put it into your inventory
OPEN(object) : open an object to search or put things in it,
               e.g., OPEN(freezer).
ACTIVATE(object) : activate / turn on an object such as sink or
                   stove, so that you can use it.
DEACTIVATE(object) : deactivate / turn off the object
EXAMINE(object) : look at an object carefully, e.g., EXAMINE(light bulb).
MOVE(object, place) : move/place the object to a place
```

In order to seamlessly integrate both the Swift and Sage modules, Lin et al. [38] developed a heuristic for interleaving the modules when the Swift module raises errors during execution. We list the conditions to switch from the Swift to the Sage module as follows:

1. Stuck: There are K=5 consecutive time steps with zero reward.
2. Invalid: The SWIFT's prediction for the next action is invalid in the current environment.
3. Critical: Next action involves a critical decision, e.g., giving the final answer for the experiment result.
4. Unexpected: The observation perceived after executing the next action suggests that an exception is encountered.

This heuristic approach aims to switch from the Swift module to the Sage module to carry on with the agent's execution, as the Sage module emulates more deliberative thinking [38]. After the agent executes all actions from the Sage action buffer, the execution alternates back to the Swift module. As the Swift module presents limitations regarding generalization, the Sage module requires the use of a more expensive LLM. Thus, the heuristic needs to interleave between both modules to address their issues.

3. AGENT ARCHITECTURE OVERVIEW

Traditional BDI approaches rely on domains encoded in a logical language, requiring human experts to design predicates and literals to implement BDI agents. In this chapter, we present a new class of agent architecture named NatBDI that applies the BDI reasoning cycle over natural language environments. Our approach encodes BDI components such as beliefs, events, intentions, and plan-rules in a natural language representation, allowing humans with virtually no programming skills to understand the agent's behaviour and develop plans.

In this chapter, we describe further the NatBDI architecture by introducing its components and explaining the agent's reasoning cycle. We organize this chapter as follows: Section 3.3 describes belief formulation from natural language observations. Section 3.4 details how the plan selection mechanism actuates over natural language. Section 3.5 introduces a component that handles failures in plan selection mechanism, named fallback policy. Finally, in Section 3.6, we describe the agent's reasoning cycle by detailing the algorithms we develop in the NatBDI interpreter.

3.1 Problem Formulation and Challenges

Text-based environment aims to evaluate the capabilities of autonomous agents to reason over natural language information and act according to a given goal [80, 28, 15]. It requires agents to comprehend natural language in order to understand aspects that comprise environment states and generate a sequence of natural language actions to perform in order to achieve the goal. In this work, we cover text-based environment implementation grounded in POMDP model (recall from Sections 2.3.3 and 2.3.2). Thus, we formally define the problem of handling a POMDP text-based environment as follows:

Definition 3.1.1 (POMDP Text-based Environment Task Problem)**.** A POMDP text-based environment task is a $\langle env, o, TD, \mathcal{R}, \mathcal{A} \rangle$, where:

- env is the text-based environment implementation;
- o is the observations perceived from initial state comprising textual sentences describing what the agent are seeing (e.g., "you see a metal pot");
- TD is the task description comprise a text explaining the details about the goal to the agent (e.g., "your task is to melt water");
- $\mathcal{R}$ is a reward function that guides the agent's behaviour;

- $\mathcal{A}$ is a set of available actions to be performed comprising in imperative sentences (e.g., “go to the kitchen”).

Most approaches employ machine learning grounded in opaque models [17] to handle text-based environments by capturing grammatical patterns in sentences that describe states and predicting actions for the agent to perform [29, 38, 12]. The objective of this thesis is to present a robust alternative for solving text-based environment tasks by developing agents that are scrutable to humans. We outline the challenges of developing scrutable agents by leveraging the Verhagen et al. [78] two-dimensional framework, which defines concepts of explainability in artificial intelligence as follows:

- **Transparency**: Transparency is a concept that brings autonomous agent the ability to represent their internal state in a way that is accessible for human scrutiny [78, 17]. We argue that this concept is essential for developing autonomous agents to address real-world problems as it enables humans to examine the agent’s behaviour during execution in an environment. Verhagen et al. [78] define the concept of transparency as “disclosing the relevant outward and functional system elements to users, enabling them to access, analyse, and exploit this disclosed information”. Autonomous agents built solely on neural networks lack transparency as their decisions rely on numerous numerical parameters with unclear interpretation.

- **Controllability**: Controllability means how human users can change or correct the behaviour of an autonomous system by instructing or introducing rules. Although Verhagen et al. [78] cover controllability concepts within transparency, we emphasize this concept by arguing that controllability guarantees the agent’s behaviour aligns with human intentions [35]. That is, in crucial situations, humans can directly determine the agent’s behaviour by specifying what actions it should perform to ensure safety and security.

- **Explainability**: Given a transparent system, the concept of explainability allows an autonomous agent to clarify its disclosed information, enabling humans to understand the causality of its behaviour. For example, humans can understand the cause behind the agent’s decisions by analysing the agent’s belief about the world (e.g., decision X caused by belief Y). According to Verhagen et al. [78] , transparency turns an *incomprehensible system* into an *interpretable system*, and explainability turns an *interpretable system* into an *understandable system*. Thus, both concepts complement each other, providing a simple but robust alternative framework to develop scrutable autonomous agents.

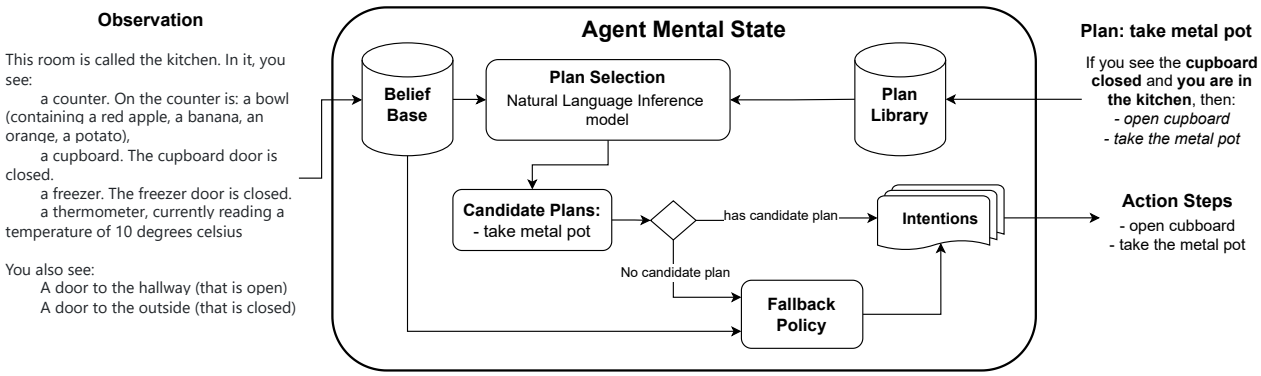


Figure 3.1 – Diagram illustrating the NatBDI architecture to handle and actuate over natural language environments.

3.2 Agent Architecture Overview

In this section, we provide an overview of our BDI agent architecture to perform tasks in text-based environment, providing a high-level overview of how we organize the BDI components within the agent's mental state to handle natural language information. In text-based environments, in the first step, the agent receives a task description consisting of a small text detailing how to achieve the goal. Such a goal indicates what the agent should do, and it is written in natural language as an imperative sentence (e.g. boil water, take metal pot, etc).

At each interaction with the environment, the agent perceives the state as a description of the agent's current location in the form of textual observations. The textual observations detail what the agent perceives from the environment, which consists of sentences that can describe objects perceived, doors contained in the room, etc. The reasoning process uses such observations as input to update its belief base, which it then uses to make inferences over its course of action. In Figure 3.1, the agent perceives that it is in the kitchen and perceives a set of objects that comprise the room. As its goal is to take the metal pot, the agent searches in its natural language plan library and finds a plan that instructs it to open the cupboard and take the metal pot, which lead toward its goal.

Once the agent perceives the goal from environment, the reasoning process starts to find candidate plans that address such goal. The plan selection mechanism searches for options consisting of plans written in natural language, that is, it represents the goal, plan context and instructions in its body as sentences. We introduce a novel component to execute logical inferences over sentences, which address the plan context verification by checking whether it logically follows the belief base (recall Section 2.4). Given a candidate plan found by the reasoning process, each step in its plan body is included in the intention structure. During the execution of the agent's reasoning process, intention failures might occur in cases where no plan has been found (i.e., no plan can-

didates) given a specific goal addition event. Since text-based environments inform the agent whether the task is finished after each action performed, the agent can perform the entire candidate plan and still fail to reach the goal provided by the environment. To deal with such cases, the agent executes an intention failure mechanism called fallback policy which we detail further in Section 3.5.

Given the components presented above, along with the new fallback policy, we extend Definition 2.2.1 to formally describe our architecture as follows:

Definition 3.2.1 (NatBDI Agent)**.** An NatBDI agent is a tuple $\langle Ag, \mathcal{E}, \mathcal{B}, \mathcal{PL}_{nl}, \mathcal{I}, \pi_{FB} \rangle$, where Ag is the agent identifier; $\mathcal{E}$ is the queue of events; $\mathcal{B}$ is the belief base consisting of natural language observations; $\mathcal{PL}_{nl}$ plan library—is a set of plan rules written in natural language; $\mathcal{I}$—the intention structure— is a set of intentions; and π_{FB} is the agent's fallback policy.

3.3 Belief Base

Our approach to representing the natural language belief base follows the traditional BDI model, which organizes the information perceived into literals in its mental model, except that our agent receives observations written in natural language instead of symbols. The natural language belief base consists of sentences written in the second person, indicating to the agent its perceptions in a human-readable format. Each belief is a sentence indicating what the agent perceives in the current turn jointly with objects currently in its inventory.

Each sentence is analogous to a symbolic belief representation from traditional BDI approaches, which expresses information such as what the agent perceives (e.g., "you see a table") or where the agent is located at (e.g., "you are in the kitchen"). Instead of relying on ground predicates with terms designed in a predetermined structure, natural language beliefs comprise in a less constrained structure. It leverages the intrinsic nature of human language to represent the same semantic meaning using different expressions (i.e., ambiguity) while turning the language processing more complex [33, 23]. We argue that representing beliefs as natural language sentences helps humans to understand what the agent has perceived about the world, allowing them to scrutinize further its mental state and have more comprehension of its behaviour.

Figure 3.2 shows a comparison of the belief base formulation process based on perceived information from the environment between traditional BDI approaches and the NatBDI belief base. In order to apply traditional BDI agents in a text-based environment, we require a language parser that follows the domain definition to identify each fact from observation and convert it into a grounded predicated [31]. Such symbols should follow a domain ontology designed manually by human experts in the environment. In contrast, NatBDI beliefs consist of sentences contained in the perceived observation, which require

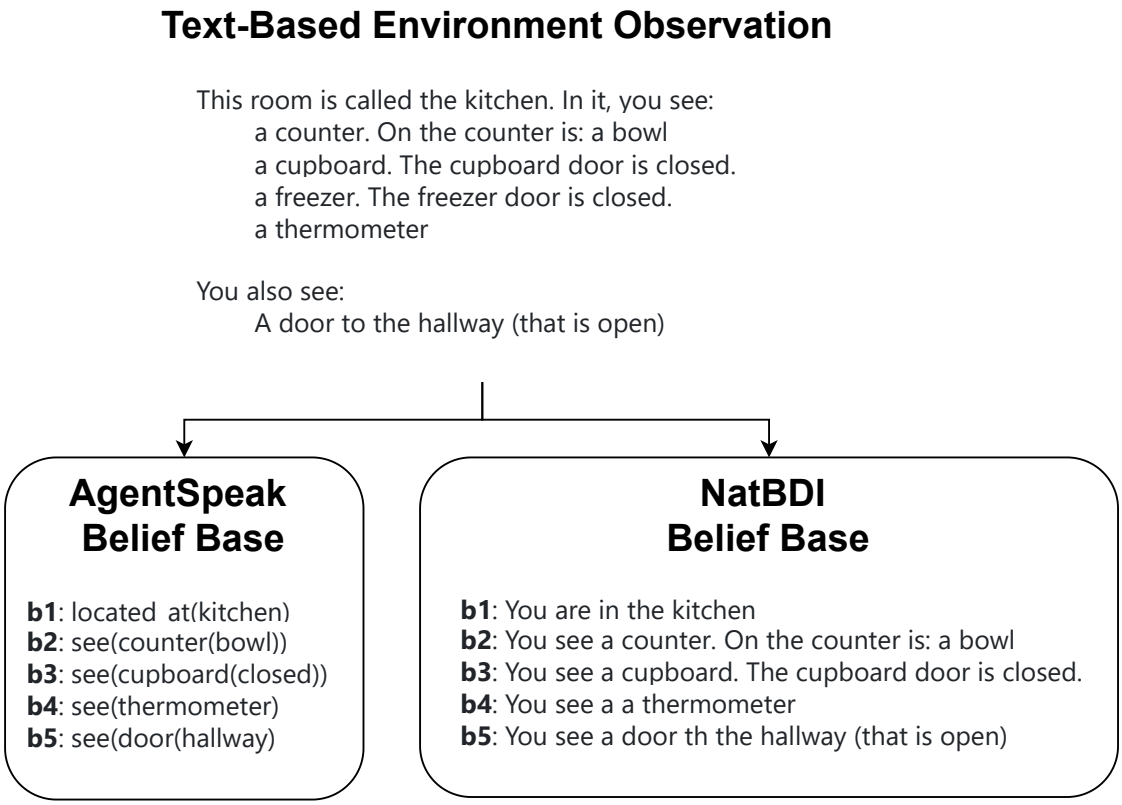


Figure 3.2 – Image comparing the belief base representation by using symbolic beliefs in AgentSpeak language and natural language beliefs by using our approach. Grounded predicates in this image are exclusively for illustrative purposes.

only manipulation of the text string by segmenting each sentence into a single belief. Our research focuses on applying the BDI reasoning process directly over natural language sentences, which does not require any specific formalism or text structure in sentences that represent beliefs.

3.4 Plan Definition

In this section, we explain how NatBDI handles natural language plans into its BDI reasoning cycle [32]. In Section 3.4.1, we detail the plan language specification to develop natural language plans in the NatBDI plan library. Section 3.4.2 details how the NatBDI selects a plan to perform, describing how the reasoning cycle infers candidate options by its plan context.

3.4.1 Natural Language Plan Library

Besides interacting with a world described in natural language, our agent architecture relies on a natural language interface for agent developers to encode plan-rules using natural language in order to facilitate the plan development for humans who are non-expert in an agent programming language (e.g., AgentSpeak [55]). We represent natural language plans in a controlled natural language [22] that contains clearly described conditions and beliefs. A controlled natural language is a subset of a natural language more amenable to automated processing, which we use to allow programmers to intersperse natural language within the structure of a plan rule. In contrast with approaches that translate the controlled natural language plans into a symbolic representation (i.e., Prolog clauses) [22], our architecture reasons directly over natural language plan-rules during plan selection.

We use this controlled natural language in NatBDI to create plans that follow a similar structure to AgentSpeak plan-rules. Thus, each plan rule consists of a statement for the intended goal, a statement of the plan context, and a set of sentences defining the plan body. The goal is a sentence describing the task that the agent intends to perform, which is analogous to the triggering event t (recall Section 2.2.2). The NatBDI selects natural language plans by matching the goal addition events from the agent's event queue with the triggering events of the plans. The plan context consists of a set of natural language sentences linked with the word "AND" to represent the logical conjunction between such statements. More formally, plans in our controlled natural language follow the template in Listing 3.1:

Listing 3.1 – Template for a plan written in NatBDI controlled natural language.

```
IF <goal statement>
CONSIDERING <plan context statements>
THEN:
<plan body>
```

In order to accept hierarchical plans, the agent interpreter accepts sentences representing actions or subgoals in the plan body. Here subgoals work similarly to goal addition events in AgentSpeak [55]. In the plan body section, we include the keyword "**PLAN TO**" to distinguish sentences that encode a goal addition, which is analogous to a goal addition event, with sentences describing actions. Much like in AgentSpeak, an agent keeps track of subgoals in a stack data structure. As the agent adopts new goals, they are stacked, and as it achieves them, they are unstacked. Such a keyword helps the agent interpreter disambiguate between actions sent to the environment, and the internal reasoning for recursive subgoals.

For example, the natural language plans described in Listing 3.2 shows how a human can explicitly instruct the agent to achieve particular tasks. In the first plan, a human knows that the metal cup is in the cupboard, which is initially closed, and instructs the agent on how to obtain the metal cup using a natural language plan. The triggering event is the goal “get the metal pot” and it should be considered as a plan candidate if the context of the plan is true, that is, if the agent is in the kitchen and it is seeing a closed cupboard. The plan body consists of two actions written in natural language instructions that will be interpreted by the textual environment. First the agent should open the cupboard, and then it should take the metal pot. The second plan triggers when the goal is to melt water. This illustrates part of a plan that requires a subgoal (get the metal pot).

Listing 3.2 – Plans in our controlled natural language to pick the metal pot and melt water in ScienceWorld.

```
IF your task is to get the metal pot
CONSIDERING you are in the kitchen
        AND you see the cupboard closed
THEN:
open the cupboard,
take the metal pot

IF your task is to melt water
THEN:
PLAN TO get the metal pot
pick up thermometer
...
```

3.4.2 Entailment with Natural Language Beliefs

Most implementations of the BDI architecture select plans based on a context condition that, when entailed by the agent’s belief base, triggers plan adoption. This creates a filter for options to be executed by the agent afterwards [18]. Concretely, if the belief base logically entails the conditions described in a plan, then the agent commits to executing its steps within an intention structure. Since the agent deals with natural language information, computing entailment within natural language becomes a key challenge. We apply a natural language inference (NLI) model grounded by machine learning to emulate the entailment operation over natural language information in NatBDI architecture [39, 83].

In the BDI model, the plan-selection mechanism assumes that the entailment inference consists of a Boolean value indicating whether a plan is candidate/applicable or not. Recent approaches develop NLI models as a three-way classification method generating the following three logical relations: entailment, contradiction, and neutral [83, 3] (recall from Section 2.1.2). Since our main objective is to infer whether the beliefs entail a specific plan context, we unify the contradiction and neutral classes as a non-entailment relation [41]. Thus, we use the NLI model as a binary classifier (i.e., returning a Boolean signal) under a closed-world assumption. Additionally, we assume the classifier returns the entailment result along with a confidence value to guide the BDI plan selection mechanism. We formally encapsulate the NLI model in a function, which we describe in Definition 3.4.1.

Definition 3.4.1 (NLI Function)**.** A *nli* function is a function $nli : (p, h) \rightarrow \langle ent, nli_{confidence} \rangle$ where p and h represents the premise and the hypothesis sentences respectively, ent is a Boolean value indicating whether p entails h, and $nli_{confidence}$ is a numerical value indicating the model inference confidence.

In practice, to infer whether the belief base entails a plan context, we need to compare if each sentence in the plan context has at least one belief that entails it. Specifically, the agent needs to compare every belief in the belief base with all plan contexts to decide if such a plan should be selected. Since beliefs and the plan context consist of multiple sentences, NatBDI processes the entailment inference between these two structures by creating the Cartesian product between both sentence sets.

Formally, given two sets of sentences representing the belief base $\mathcal{B}$ and $\mathcal{C}$ respectively, we create a matrix represented as $\mathcal{C} \times \mathcal{B}$ in Equation 3.1. For each belief $b_i \in \mathcal{B}$ and plan context $c_i \in \mathcal{C}$ pair contained in the $\mathcal{C} \times \mathcal{B}$, we employ the natural language inference represented as *nli* function (Definition 3.4.1) in Equation 3.2 resulting in a Boolean matrix $E_{i,j}$. Matrix $E_{i,j}$ contains all inference results for each i-th plan context and j-th belief.

$$M_{i,j} = \mathcal{C} \times \mathcal{B} = \{(c_i, b_j) \mid c \in \mathcal{C} \wedge b \in \mathcal{B}\} \tag{3.1}$$

$$E_{i,j} = \{nli(c_i, b_j) \mid (c_i, b_j) \in M_{i,j}\} \tag{3.2}$$

To check whether a plan is a candidate to be selected by the agent, we apply the disjunction given the Boolean values in $ent_{i,j}$ for each i-th context with all j-th beliefs. Given each Boolean generated by the disjunction, we apply the conjunction for each c resulting in a single Boolean value. Thus, we define entailment of a plan context from the belief base in NatBDI as in Equation 3.3.

$$\mathcal{B} \models \mathcal{C} \doteq \bigwedge_{c_i \in \mathcal{C}} c_i \bigvee_{b_j \in \mathcal{B}} b_j \tag{3.3}$$

For example, consider a state with a belief base that contains the following sentences:

- “this room is called the kitchen”
- “you see a cupboard, the cupboard door is closed”

An inference between such belief base and the plan with context composed of the sentences

- “you are in the kitchen”
- “you see a closed cupboard”

works as follows. First, we compute the values of matrix $E_{i,j}$ with i-th row representing context sentences and j-th column representing belief sentences as follows:

$$\begin{bmatrix} T & F \\ F & T \end{bmatrix} \tag{3.4}$$

Belief “this room is called the kitchen” entails context “you are in the kitchen” while “you see a cupboard, the cupboard door is closed” entails “you see a closed cupboard”. The first disjunction operation between all beliefs with each context results in the following matrix:

$$\begin{bmatrix} T \\ T \end{bmatrix} \tag{3.5}$$

and, consequently, the conjunction operation results in the scalar Boolean $\begin{bmatrix} T \end{bmatrix}$, which is the final result of the entailment inference, and therefore we can conclude that the information in the belief base entails the plan context. Consequently, this is a candidate plan.

3.5 Fallback Policy

When the agent retrieves a goal addition event that does not correspond to the triggering condition of any plans in the plan library, the traditional BDI reasoning cycle raises a failure during plan selection, as we describe in Section 2.2. In text-based environments, developed plans might not achieve the task goal even when executing successfully every plan step due to a failure in the plan design [43]. In our agent architecture, we introduce an approach to address plan failures in cases where a goal addition event does not correspond to any plan-rule in the agent’s plan library, which we call the fallback policy.

We automatically generate the fallback policy by employing reinforcement learning methods on natural language domains, which use prior experiences to map the text state to an action. Recall from Section 2.3.3 that approaches that apply reinforcement learning on text-based environments [28, 80, 29, 38] model the agent’s decision-making

component as a POMDP. We incorporate existing RL approaches encapsulating the trained policy into a function to predict an action given the current belief base, which we formally detail in Definition 3.5.1. Our approach leverages the correspondence between POMDP belief state bs and BDI belief base $\mathcal{B}$ to design the function π_{FB}[63]. Conceptually, our architecture is agnostic of any specific policy implementation, which allows integrating different methods and comparing them in our experiments.

Definition 3.5.1 (Fallback Policy)**.** The fallback policy is a function $\pi_{FB} : \mathcal{B}_t \rightarrow a_t \in \mathcal{A}$ that maps the belief base $\mathcal{B}_t$ to an atomic action a_t in turn t.

In our approach, each action a predicted by the fallback policy works the same way as a BDI action since the reasoning cycle handles them uniformly. Notice that the fallback policy produces only atomic actions, that is, it does not support generating events such as goals addition events to trigger other plans in the agent's plan library.

3.6 NatBDI Interpreter

In this section, we introduce the NatBDI interpreter to handle text-based environments detailing all mechanisms involved in the BDI reasoning cycle. We organize this section as follows: First, we describe the reasoning cycle loop adaptation to handle text-based environment goals in Section 3.6.1. Second, we describe how the interpreter handles the action execution and what objects the environment returns in Section 3.6.3. Third, we detail our approach to control events perceived from the environment within the interpreter in Section 3.6.2. Finally, we explain the fallback mechanism in cases where the interpreter raises an intention failure in Section 3.6.4.

3.6.1 Control Cycle

In this section, we describe the BDI reasoning cycle that controls the agent behaviour in text-based environments. In contrast to traditional BDI interpreters, which assume that a successful plan execution achieves the main goal, the NatBDI interpreter distinguishes between the plan goal and the environment goal. Text-based environments[28, 80] provide a text instructing the agent to accomplish the task objective. Developed plans might contain design errors that can lead to a successful execution in the reasoning cycle while failing to accomplish the environment goal. Algorithm 3.1 shows a pseudocode describing the NatBDI interpreter reasoning cycle, which receives the BDI agent tuple that follows Definition 3.2.1 along with a text-based environment instance env.

To start the reasoning cycle, we encode the task description text provided by the environment into a goal addition event, considering that the agent has a plan to handle

1: **procedure** NatBDIInterpreter($\langle Ag, \mathcal{E}, \mathcal{B}, \mathcal{PL}, \mathcal{I}, \pi_{FB} \rangle$, *env*)
2: $\quad \mathcal{E} \leftarrow$ *perceiveObjective*($env, \mathcal{E}$)
3: $\quad$ *finished* $\leftarrow$ `False`
4: $\quad$ **while** is not *finished* **do**
5: $\quad\quad \mathcal{B} \leftarrow$ *updateBeliefs*($\mathcal{B}, \mathcal{E}$)
6: $\quad\quad \mathcal{E}, \mathcal{I} \leftarrow$ *selectPlans*($\mathcal{E}, \mathcal{B}, \mathcal{PL}, \mathcal{I}, \pi_{FB}$)
7: $\quad\quad \mathcal{E}, \mathcal{I}$, *finished* $\leftarrow$ *executeIntentions*($\mathcal{E}, \mathcal{I}, env$)

Algorithm 3.1 – Abstract implementation of the NatBDI interpreter based on the BDI interpreter from Algorithm 2.1

the task in its plan library. Notice that perceiveObjective is outside the control loop since the interpreter assumes that the environment provides a single task to accomplish. The instruction encoding process depends on the text-based environment implementation since there are different approaches to writing the task description, and we show examples of task descriptions converted into BDI goals in Section 4.2. For clarity purposes, we assume there exists a function perceiveObjective that handles the task description in Line 2 of Algorithm 3.1.

In Lines 5 and 6 of Algorithm 3.1, the interpreter calls the functions updateBeliefs and selectPlans mentioned in Section 2.2.2, which follows the same underlying logic of Algorithm 2.3 and 2.5, respectively. However, since NatBDI handles both beliefs and plans in natural language, we adapt the implementation of both functions as we detail in Section 4.2 for a specific text-based environment.

Recall that the interpreter assumes the task is not yet accomplished after the agent executes all intentions from the plan successfully. Considering this, we introduce the Boolean variable *finished* to control the BDI reasoning cycle loop that text-based environments provide to the agent after performing an action. As mentioned in Section 2.3.3, the interpreter considers that the text-based environment implementation returns a signal indicating whether the agent has accomplished the main task, after executing its actions. Although we conduct our experiments in a specific environment (Chapter 4), NatBDI architecture is generalizable to other text-based environments as long as they match the Definition 3.1.1. Thus, function executeIntentions (Algorithm 3.1, Line 7) controls this variable throughout the execution of all intentions, which we describe further in Section 3.6.3.

3.6.2 Events

The NatBDI interpreter follows the BDI model approach to represent relevant perceptions for the agent as events, thereby controlling the reasoning cycle. Each time the agent performs an action, the text-based environment sends back to the agent the up-

dated version of the current state resulting from the action effects. The text informing the updated scene covers the updated observations along with unmodified ones, which helps to maintain the agent's perception coherence without any specific processing to compare the previous observations with the new ones [64]. Thus, we simplify the maintenance of the NatBDI natural language belief base throughout the reasoning process in text-based environments by merging the semantics of belief addition and deletion events into a belief update event.

The belief update event carries a list of new beliefs that the agents perceive after performing actions in the environment. The updateBeliefs function updates the belief base by retrieving the belief update event and overwriting completely the entire belief base with the new perceived beliefs. Notice that the NatBDI interpreter does not have any sophisticated belief revision mechanism, as it assumes that the environment is responsible for maintaining the coherence of the information the agent perceives while actuating in the environment. This feature of text-based environments [80, 15, 28] enables the agent to bypass the frame problem [64], as it represents the environment information, including both modified and unchanged elements resulting from the actions performed.Thus, overwriting the entire agent's belief base is a simple yet effective approach to preventing inconsistencies, particularly when the effects of performed actions might lead to discrepancies in the belief base. We formalize the belief update event by extending Definition 2.2.3 as follows:

Definition 3.6.1 (Belief Update Event)**.** A belief update event, denoted as $\pm\varphi$, updates the belief base $\mathcal{B}$ by overwriting the existing beliefs with the observations φ.

For example, if the agent moves from the location loc_a to loc_b, it requires the interpreter to remove all beliefs that represent the objects perceived from the loc_a since the agent is not observing them currently and to include the beliefs perceived from loc_b in the belief base. The environment returns a new text informing the scene perceived exclusively from loc_b from which the interpreter formulates a new set of beliefs to overwrite the current belief base. A potential inconsistency this example might bring to the belief base is including the belief "you are in loc_a" along with the existing belief "you are in loc_b" in the same turn, which would inform the agent that it is in two locations at the same time. The interpreter prevents this belief incoherence during the execution of updateBeliefs function (Algorithm 3.1, Line 5) by deleting "you are in loc_a" and adding "you are in loc_b" in the belief base due to the state information provided by the environment to the agent already accounts for these changes.

3.6.3 Plan Selection

In the BDI model, agents actuate over the environment by selecting plans in their plan library, and executing the steps in the plan body in accordance with the plan developer instructions. In this section, we refine our approach to carry out both operations in an abstract NatBDI interpreter implementation for text-based environments. We leverage the underlying logics of selectPlans and selectOptions mentioned in Section 2.2.2, adapting such functions to use the NatBDI architecture components.

The mechanism for selecting options queries the plan library to find candidate plans, relying on the inference of entailment relations between the beliefs in the belief base and the statements that constitute a plan's context. We refine the selectOptions function (Algorithm 2.4) to introduce the entailment mechanism over natural language sentences following Definition 3.4.1. The interpreter applies the *nli* function by using all beliefs in the belief base as premises and the plan context as hypothesis, and we detail the inference mechanism further in Section 3.4.2. The list of candidate plans that this operation aims to achieve contains the plan rule found, along with the inference confidence to steer the plan selection mechanism. Notice that both beliefs and plan contexts are grounded sentences, which neither support the *unify* operation (Line 4 in Algorithm 2.4) nor substitutions (Line 4,5,6 in Algorithm 2.4) that handle lifted predicates. We describe the adapted version of *selectOptions* in Algorithm 3.2.

```
1: function selectOptions(e, B, PL)
2:     Opt ← ∅
3:     for plan ⟨t, C, bd⟩ ∈ PL do
4:         ent, nli_confidence ← nli(B, C) // NLI function from Definition 3.4.1
5:         if ent is true then
6:             Opt ← Opt ∪ ⟨plan, nli_confidence⟩
       return Opt
```

Algorithm 3.2 – Option Selection algorithm adapted from Algorithm 2.4 to select natural language plans from the NatBDI's plan library,

Given the use of NLI function (Definition 3.4.1) in the option selection function, we adapt the plan selection mechanism to leverage the results from the inference mechanism to rank plans. Specifically, the plan selection mechanism selects the plan with the highest confidence ($nli_{confidence}$) as the plan to be performed within the BDI reasoning cycle. In Algorithm 3.3, we describe the adjustments to support the inference over natural language, where we modify Line 6 of Algorithm 2.5 to specify the option selection criterion (i.e. Line 5 of Algorithm 3.3) in cases where there are more than a single candidate. Recall that the interpreter does not work with belief and plan contexts that use lifted representations and, hence, we remove the substitutions (σ) originally included in selectPlans of Algorithm 3.3.

To handle failed intentions regarding plan selection, we replace the intentionFailure (Line 10 of Algorithm 2.5) function with the fallback policy function, which we explain further in Section 3.5.

```
1: function selectPlans(e, B, PL, I, π_FB)
2:     I' ← ∅
3:     E', e ← pop(E)
4:     Opt ← selectOptions(e, B, PL)
5:     if Opt ≠ ∅ then
6:         Pick the plan rule option ⟨t, C, bd⟩ with the highest nli_confidence.
7:         I' ← addIntention(e, bd, I)
8:     else
9:         I' ← fallbackPolicy(B, I, π_FB)
       return E', I'
```

Algorithm 3.3 – Plan selection algorithm adapted from Algorithm 2.5 to retrieve candidate options from the NatBDI's plan library.

As mentioned in Section 3.6.2, the interpreter updates the belief base after the agent performs an action by using the environment's feedback, which provides a textual description of the agent's observation considering the effects caused by its action. In cases where the step in the intention stack represents an action, it requires adding the belief update event following the execution of an action. Considering this, we introduce Algorithm 3.4 to detail an implementation adapted from Algorithm 2.8 of intention execution on text-based environments. The NatBDI interpreter uses the feedback provided by text-based environments, which we explain in Section 2.3.3, using the Boolean variable *finished* to finish the loop inside the control cycle (Algorithm 3.1) and the observations after the agent performs an action. In this adapted version for NatBDI, we do not cover belief addition/deletion events since the textual plan language only covers steps that comprise actions and goal addition events in the plan body.

3.6.4 Fallback Policy as Intention Failure Handling

In this section, we introduce an abstract implementation of the fallback policy mechanism within the NatBDI interpreter (Algorithm 3.1). We use the fallback policy from Section 3.5 as an intention failure mechanism to infer actions without developed plans. The agent then employs the fallback policy to handle scenarios where there is no plan candidate available in the agent's plan library.

Initially, as the plan selection mechanism selects candidate options by analysing the belief base, the fallbackPolicy function maps the belief base into a fallback action a_{fb}. The interpreter instantiates a new intention stack st'_{fb} to formulate a new intention and includes it into the agent's intention structure $\mathcal{I}$. Thus, the NatBDI agent executes the

```
function executeIntention(I, E, B, env)
    E', I' ← ∅
    int ← topmost intention from I
    int', s ← pop([s_1, ..., s_n])
    if s is an action then
        o, R, finished ← executeAction(s, env)
        ±b ← belief update event containing new beliefs from o (Definition 3.6.1)
        E' ← push(±b, E)
        if int = [] then
            I' ← I − {int}
            return E', I'
    else if s is a subgoal g then
        E' ← push(+g, E)
    I' ← (I − {int}) ∪ {int'} //removes the old intention and includes the new one.
    return E', I', finished
```

Algorithm 3.4 – Function to instantiate intentions given a new intention stack with all substitutions applied [43].

action a_{fb} using the same approach as actions from the manually developed plans do, since the intention structure $\mathcal{I}$ consists of both action types.

```
function fallbackPolicy(B, I, π_FB)
    a_fb ← π_FB(B) //action predicted by the fallback policy
    st_fb = ∅
    st'_fb = push(a_fb, st_fb) // new intention stack containing the fallback action
    I' ← I ∪ {st'_fb}
    return I'
```

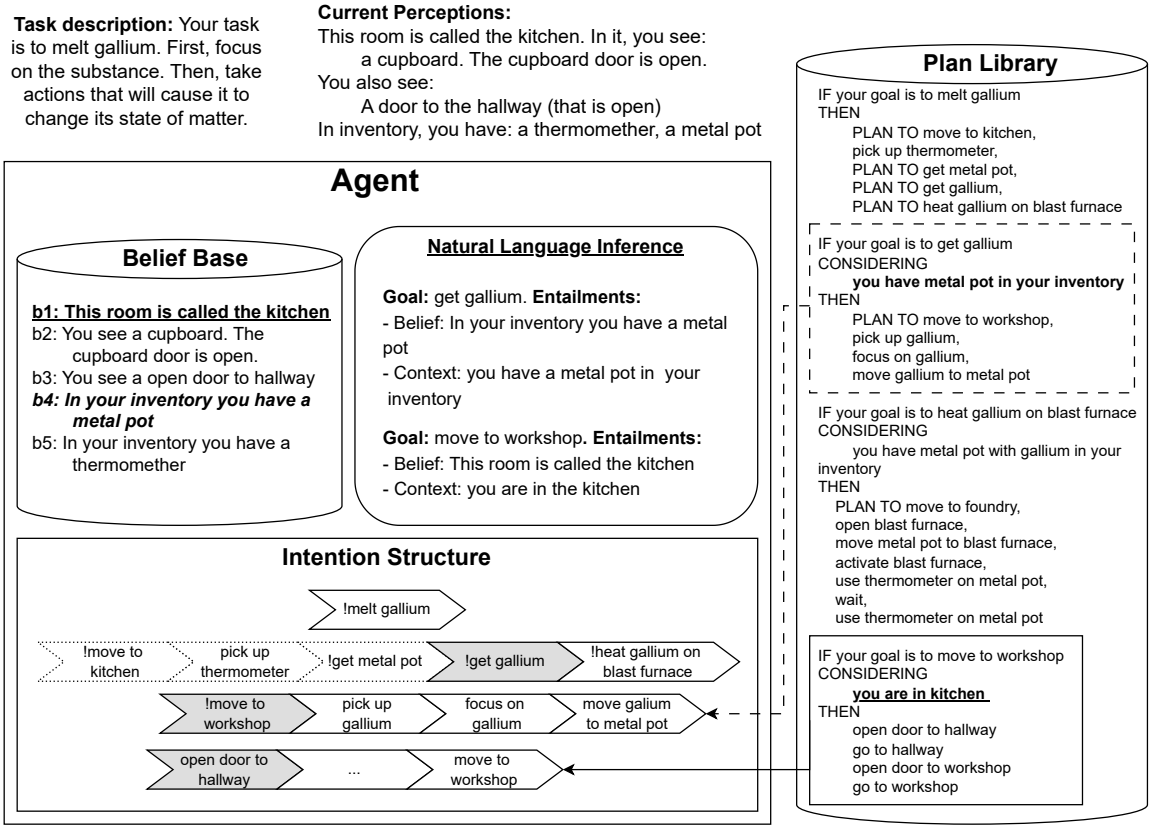
Algorithm 3.5 – Fallback policy function that includes the fallback action a_{fb} into the intention structure $\mathcal{I}$.

3.7 NatBDI Execution Example

In this section, we describe a concrete example of NatBDI execution in a text-based environment to detail how the interpreter executes the reasoning cycle. Figure 3.7 illustrates an example in which the NatBDI reasoning cycle instantiates plans to address the task and shows how the NLI component performs inferences during plan selection. Recall from Section 3.1 that the agent receives the task description, which represents a text instructing the agent how to achieve the goal and the observation perceived by the agent in the initial state. This example illustrates the NatBDI addressing the “melt” task from the ScienceWorld environment with a plan library containing plans to solve it. We illustrate an intermediate step in the execution of the reasoning cycle based on an

instantiated plan, with the already executed steps represented by dashed lines and the current steps by filled figures.

Figure 3.3 – NatBDI reasoning cycle example with plans addressing the ScienceWorld “melt” task.



In this example, the interpreter parses the text from the task description into a single sentence, which we detail in Section 4.2, in order to facilitate the plan development. The main goal “melt gallium” triggers a plan, shown in dashed lines, that populates the intention structure with five steps, and Figure 3.7 illustrates the execution of the “get gallium” step, which is a goal addition. Notice that the interpreter selects the second plan (with the “get gallium” goal) as the option to execute because the belief “In your inventory, you have a metal pot” entails its plan context (i.e., “you have a metal pot in your inventory”). We illustrate in bold the beliefs from the belief base that entail the plan context.

The first step in the plan body for handling the “get gallium” goal instantiates another plan, extending the intention structure with 4 additional steps. That is, to obtain the gallium substance, the agent needs to go to the workshop room (fourth plan), which triggers a plan to move from the kitchen to the workshop. In this case, the plan from “move to workshop” includes the action “open door to hallway” into the agent’s intention structure, indicating the first instantiated action from the “get gallium” plan. We illustrate in the underlined text the belief that supports this option.

4. EXPERIMENTS

In this chapter, we describe our experiments to evaluate NatBDI in reasoning tasks within a textual environment. First, we detail the implementation details of NatBDI architecture to perform our experiments in Section 4.1.1. Second, we specify the text-based environment that we use in our experiments in Section 4.2. Third, we describe the process of creating natural language plans for performing our experiments in Section 4.3. Fourth, we specify the existing approaches that we use as a fallback policy component in our experiments in Section 4.4. Finally, we detail the results of the experiments in Section 4.5.

4.1 NatBDI Agent Implementation

In this section, we report details on the implementation the NatBDI for conducting our experiments. First, we describe the development of the NatBDI implementation in Section 4.1.1. Second, we outline our matrix-based implementation for performing logical inference using NLI in Section 4.1.2.

4.1.1 Implementation Details

We developed the NatBDI components using the Python language in order to integrate with the most recent neural network libraries. We implemented this operation by using a neural network library, pytorch [1], and employed a pretrained language model from the HuggingFace repository [87]. Our developed code is available in the following GitHub repository: https://github.com/yukioichida/nat-bdi.

Both the event queue and intention structure comprise a Deque[2] structure from Python collection. The agent's event queue uses operations such as *appendLeft* and *popLeft* to represent a first-in-first-out structure, whereas the intention structure uses stack operations to represent a last-in-first-out structure. Recall from Algorithm 3.4 that the body of each plan contains an intention stack with the steps for the agent to perform. Thus, during the agent reasoning cycle process, we develop the intention structure as a single Deque containing all plan bodies from selected plans concatenated in a contiguous structure.

In our experiments, we use an existing neural network pretrained on NLI annotated datasets which contains optimized weights openly available in the HuggingFace plat-

[1] https://pytorch.org/
[2] https://docs.python.org/3/library/collections.html#collections.deque

form. Specifically, we implement the NatBDI agent using a *roberta-large*[39][3] language model trained on the MultiNLI dataset (MNLI)[83]. For each sentence pair belief-plan context, our implementation concatenates both sentences as input to the language model, which outputs the class probabilities (i.e., entailment and not-entailment) and predicts the logical relationship by selecting the class with the highest probability (i.e., greedy selection). Recall from Definition 3.4.1 that NatBDI requires a confidence value to prioritize options when multiple plans are available. Since we employ a neural network where its output layer returns the entailment probability, we calculate the average probability of all sentences classified as entailment and adopt it as a confidence value for plan selection.

We execute our experiments in a machine with 12 core AMD Ryzen 5 with 32GB of RAM, and an NVIDIA RTX 3060 GPU. In this hardware specification, the execution time of NatBDI implementation is around 3 seconds per test variation. We define 50 as the maximum limit for fallback actions to avoid infinite loops in the NatBDI reasoning cycle.

4.1.2 Plan Selection Inference

In this section, we explain the implementation details of the plan selection mechanism that NatBDI uses for filtering options in the plan library. Recall from Section 3.4.2 that NatBDI architecture employs a natural language inference model to infer the entailment relation between the agent's belief base and a plan context formula. Concretely, we develop a matrix-based implementation of the selectOptions function (Algorithm 3.2) that executes a pretrained neural network perform logical inferences for each sentence pair (p, h).

To detail further the implementation of the NatBDI plan selection, let us consider an example of a belief base containing the following beliefs:

- "This room is called the kitchen"
- "You see a cupboard. The cupboard is closed"
- "You see a door to the hallway (that is open)"

and a plan context containing the following statements:

- "you see the cupboard closed"
- "you are in the kitchen"

Below, we enumerate the steps that the matrix-based implementation of the *selectOptions* function performs for this example:

[3] https://huggingface.co/FacebookAI/roberta-large-mnli

1. **Allocate the matrix M to represent all belief, context sentence pairs:** The function *selectOptions* instantiates the matrix M with each row representing all beliefs from belief base and each column representing statements from plan context. We formalize the matrix M in Equation 3.1 and illustrate this example in Figure 4.1.

	Plan Context	
Belief Base	you see the cupboard closed	you are in the kitchen
This room is called the kitchen		
In it, you see a cupboard. The cupboard is closed.		
You see a door to the hallway (that is open)		

Figure 4.1 – Matrix M consisting of every belief-plan context pair to be inferred.

2. **Perform *nli* inference to generate the matrix E.** In this step, given the matrix M instantiated with all belief-plan context pairs, the function calls the NLI model inference in a batch mode to efficiently use the GPU. That is, the function generates the matrix E (recall Equation 3.2) by filling the matrix M with the results of the NLI neural network batch inference. We illustrate this execution in Figure 4.2.

	Plan Context	
Belief Base	you see the cupboard closed	you are in the kitchen
This room is called the kitchen	False	True
In it, you see a cupboard. The cupboard is closed.	True	False
You see a door to the hallway (that is open)	False	False

Figure 4.2 – Matrix E with the boolean results of NLI model inference for each belief-context sentence pair.

3. **Check whether the belief base entails the plan context using the matrix E.** In this step, the function performs boolean operations on the NLI results from matrix E to infer whether the plan, given its plan context, is a valid candidate option. Recall from Equation 3.3 that the function selectOptions performs a disjunction and a conjunction operation over the matrix E. Concretely, the function executes the column-wise disjunction operation to check whether there is a belief sentence that entails the j-th context and, afterwards, a row-wise conjunction to verify whether all plan context are entailed by the belief base. We illustrate the execution of this step in Figure 4.3. Each plan context is entailed by at least one belief and, hence, this plan is a valid candidate option for the agent to select.

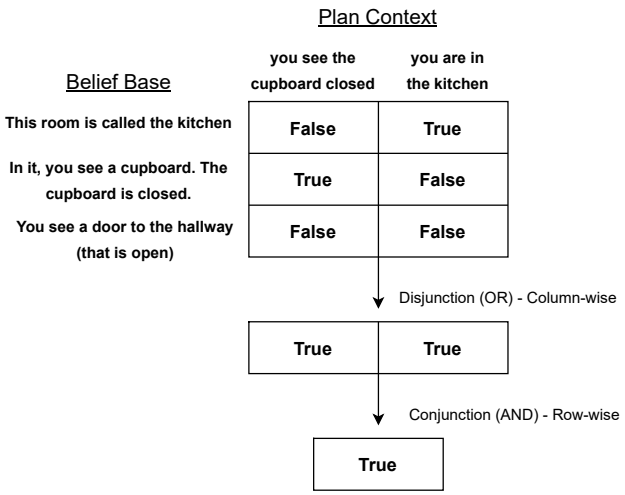


Figure 4.3 – Boolean operation performed to check if the belief base entails the plan context.

4.2 ScienceWorld Environment

In this work, we perform all our experiments with the NatBDI implementation using the ScienceWorld environment [80]. We chose to use the tasks *find-non-living-thing* and *melt* to conduct our experiments, as these are the tasks on which existing approaches achieve the highest and the lowest scores, respectively, according to Wang et al.[80]. Our objective is to evaluate NatBDI on tasks considering different levels of difficulty. In our experiments, we focus on evaluating the agent across the test variations of each task, given that such variations contain scenarios unseen in the fallback policy training step (i.e., out-of-distribution states).

Recall from Section 2.3.4 that the ScienceWorld environment encodes information about the state in a paragraph that mentions all objects within the room where the agent is present. Given that the paragraph follows a clear format to describe the current scene for the agent, we develop a simple parser to segment the paragraph into multiple sentences, each of which describes a single object to represent a natural language belief, as we detail in Section 3.3. We develop a similar approach to extract the goal to represent the goal as a single sentence, where we extract the sentence part that describes the goal by using predefined parser rules.

The ScienceWorld task *find-non-living-thing* requires the agent to find an inanimate object and place it in a container, as the task description specifies. This task aims to evaluate reasoning capabilities related to object comprehension in order to identify a non-living object, and spatial perception to find the target container to place the object. The Scienceworld environment contains 75 variations of *find-non-living-thing* task, each with different container types (e.g., red box, blue box, etc.), and the placement of the containers changes for each variation. In this task, the environment generates positive

rewards only in three situations: when the agent identifies and picks up a non-living object, moves to the target container location, and places the object in it. In Table 4.1, we show examples detailing how ScienceWorld describes the task description along with the parsed text to represent the BDI goal.

Table 4.1 – Examples of variations for the ScienceWorld task *find-non-living-thing*.

Variation	**Task Description**	**BDI Goal**
225	Your task is to find a(n) non-living thing. First, focus on the thing. Then, move it to the orange box in the living room.	find a(n) non-living thing by moving it to the orange box in the living room
226	Your task is to find a(n) non-living thing. First, focus on the thing. Then, move it to the yellow box in the living room.	find a(n) non-living thing by moving it to the yellow box in the living room
227	Your task is to find a(n) non-living thing. First, focus on the thing. Then, move it to the purple box in the living room.	find a(n) non-living thing by moving it to the purple box in the living room
228	Your task is to find a(n) non-living thing. First, focus on the thing. Then, move it to the red box in the bedroom.	find a(n) non-living thing by moving it to the red box in the bedroom
229	Your task is to find a(n) non-living thing. First, focus on the thing. Then, move it to the green box in the bedroom.	find a(n) non-living thing by moving it to the green box in the bedroom

In contrast to *find-non-living-thing* task, machine learning approaches that use ScienceWorld as environment achieve low scores on the *melt* task. In the melt task, the agent's objective is to melt a substance that the task description specifies, requiring the agent to understand how to measure the substance's temperature and act according to its melting point. The ScienceWorld environment provides 9 test variations covering 5 substances with different melting points. This task imposes a challenge for machine learning approaches because test variations involve different substances than those seen in the training variations, requiring handling substances with different melting points than those seen in the training phase. In Table 4.2, we show examples detailing the task description for *melt* task along with sentences in which the BDI agent uses them as goals.

4.3 Natural Language Plans Setup

To conduct our experiments and demonstrate the reasoning cycle process in a natural language environment, we introduce natural language plans to solve tasks and

Table 4.2 – Examples of variations for the ScienceWorld task *melt*.

Variation	Task Description	BDI Goal
21	Your task is to melt lead. First, focus on the substance. Then, take actions that will cause it to change its state of matter.	melt lead
22	Your task is to melt tin. First, focus on the substance. Then, take actions that will cause it to change its state of matter.	melt tin
24	Your task is to melt mercury. First, focus on the substance. Then, take actions that will cause it to change its state of matter.	melt mercury
26	Your task is to melt gallium. First, focus on the substance. Then, take actions that will cause it to change its state of matter.	melt gallium

incorporate them into the agent's plan library. We manually create such plans using the controlled natural language from Section 3.4.1 by analysing the task objective and defining a sequence of steps, which we empirically evaluate in the ScienceWorld environment. We leverage annotated trajectories and organize them in subsequences to evaluate how the agent operates with hierarchical instructions by handling subgoals (i.e., goal addition events). Working with subgoals allows the plan developer to reuse existing plans and introduce plans with smaller plan bodies.

To ease the manual process of creating plans for experiments, we generate plans for navigating the environment's rooms automatically by exploring and memorizing the transitions made. Specifically, we develop a function to perform random movement actions and collect the traces to identify spatial information about the environment's room. To represent this spatial information, we build a graph using nodes to represent the rooms and edges to link them to their neighbouring rooms, which we illustrate an example from ScienceWorld in Figure 4.4. We use this graph to automatically generate navigation plans by leveraging all available paths given a specific location, which we show an example of a plan generated in Listing 4.1.

Listing 4.1 – Plan to move from art studio to kitchen considering the graph from Figure 4.4.

```
IF your task is to move to kitchen
CONSIDERING you are in the art studio room
THEN:
go to hallway,
go to kitchen,
```

The navigation plans facilitate the development of plans for our experiments, which is a manual process. Such plans allow developing plans by including movement instructions to a specific room without prior knowledge of the current room. For example, Listing 4.2 illustrates a manually written plan to melt caesium, which includes a step in

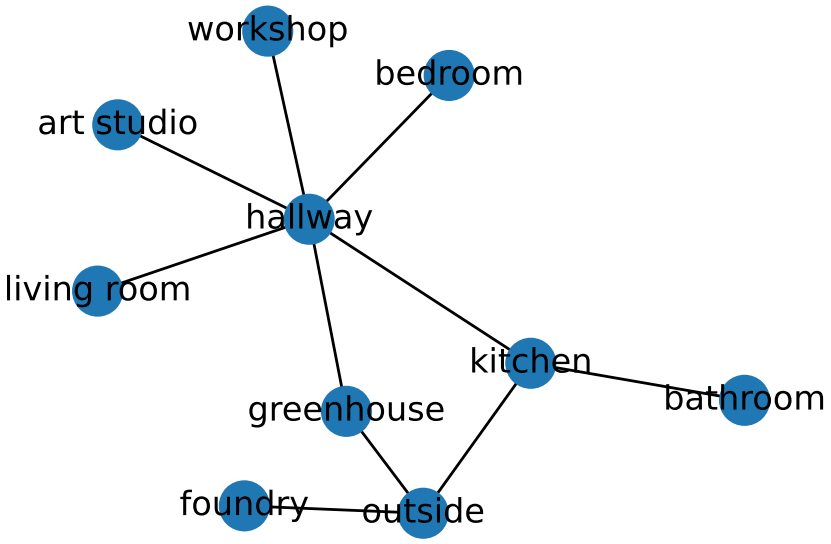


Figure 4.4 – Graph with rooms obtained from random explorations in the ScienceWorld environment.

the intention structure to generate a goal-addition event for finding a navigation plan that moves the agent to the kitchen, where the metal pot is located. As we mentioned above, the navigation plan generation creates plans for all possible paths, ensuring that the plan library contains plans across all rooms in the graph. This guarantees that the goal addition *move to kitchen* triggers a navigation plan, regardless of the agent's current location.

Listing 4.2 – Plan to melt caesium that calls a navigation plan to go to kitchen.

```
IF your goal is to melt caesium THEN
PLAN TO move to kitchen,
pick up thermometer,
PLAN TO get metal pot,
PLAN TO get caesium,
PLAN TO heat caesium on blast furnace
```

For the task "find-non-living-thing", as the task is more trivial and requires smaller and simpler plans, we develop a single plan for every variation in our experiments. In contrast, for the task "melt", as the task requires more sophisticated and complex steps such as manipulating different substance types and measuring the melting point, we introduce

plans to handle seven out of the nine variations. We note that, in ScienceWorld, it is technically possible to manually generate all possible plans by anticipating every scenario in the environment, making the achievement of top scores trivial. However, we assume that a potentially incomplete plan library is a more realistic setting, as we would not expect the developer to be able to create all possible plan-rules. When there is no candidate plan available in the agent's plan library, we evaluate the effects of using the fallback policy in such scenarios.

4.4 Fallback Policy Setup

In this section, we introduce the two fallback policy implementations that we use in our experiments in the ScienceWorld environment detailing how we adapt existing reinforcement learning approaches to use them in NatBDI architecture. We develop different implementations of the fallback policy component based on two existing machine learning approaches to evaluate their application interleaved with natural language plans in our experiments. Concretely, we choose the DRRN [29] (Section 2.4.4) and Swift [38] (Section 2.4.5) approaches, adapting them to execute seamlessly within the agent's reasoning cycle.

Recall from Section 3.5 that a fallback policy predicts an action that the interpreter includes in its intention structure. The DRRN follows the same approach in which the trained policy predicts a single action, assuming that the environment provides a set of valid actions to the agent. To generate the input text format consistent with DRRN's expected text format, in each turn, we store the original ScienceWorld texts (i.e., before parsing to beliefs) in the belief base to use them as DRRN network input. We follow Wang et al. [80] to train the DRRN in order to replicate exactly the same approach for training in the ScienceWorld environment[4].

We use the Swift Module from the SwiftSage work [38] as the fallback policy, because of its superior performance in the ScienceWorld environment compared to other machine learning approaches. Specifically, we employ the pretrained weights from HuggingFace [87][5] to perform experiments without reproducing the entire Swift training process, as we lack sufficient hardware resources for finetuning the Swift model. In this work, we use the Swift Module in isolation from the Sage module since the latter requires ChatGPT-4 [7, 89], which is neither openly available nor a free model for conducting our experiments. Similar to how we employ a DRRN as a fallback policy, we adapt the belief base to formulate the input prompt by including past information into the NatBDI's belief

[4]Available in https://github.com/cognitiveailab/drrn-scienceworld. Last accessed on March 10, 2025

[5]Pretrained model available in: https://huggingface.co/yuchenlin/swift_sw. Last accessed on March 10, 2025

base to enable instantiating a valid input prompt for the Swift model, as we describe in Listing 2.1.

We follow the DRRN implementation from Wang et al.[80], which generates a distinct trained model for each task type. In contrast to the DRRN training process, Lin et al. [38] trains the Swift module by using the same language model instance for all ScienceWorld tasks. Thus, we use the same Swift pretrained model for all experiments. While DRRN uses a smaller network with 1.5 million parameters, the Swift module comprises in a pretrained T5 language model that contains 770 million parameter, which is significantly larger than DRRN neural network.

4.5 Experiment Results

In this section, we show the results in which we obtain from our experiments. Section 4.5.1 shows the results of NatBDI on ScienceWorld, detailing its scores for each task and distinguish results obtained from plans and fallback actions. Section 4.5.2 reports the scores from fallback policy by analysing the effects of including training episodes. Section 4.5.3 reports the results in which we obtain by using different NLI pretrained models.

4.5.1 NatBDI Results in ScienceWorld Environment

We now describe the scores that our natural language BDI agent obtains using the ScienceWorld environment. First, we measure the BDI agent performance when varying the number of plan-rules in the agent's plan library for the Scienceworld tasks "find-non-living-thing" and "melt" (recall from Section 4.2) we selected from the ScienceWorld environment. We show the effects in the overall score and its progress by adding natural language plans designed by humans. Second, we evaluate the trade-off between plan-rules and the two different fallback policies, and the impact that each one can have depending on the size and quality of the plan-rules library. Finally, we evaluate using both fallback policies by itself, that is, an agent with no plan-rules that relies only on the fallback policy.

Table 4.3 shows the scores that the agent obtains in "find-non-living-thing" and "melt" ScienceWorld tasks using different fallback policy implementations (i.e., Swift and DRRN). We detail different executions varying the number of plans included in the NatBDI plan library (Num Plans). The total score represents the accumulated reward received by the agent averaged over a number of variations of the task in the same environment, which we then break down the score obtained from using plan-rules (BDI) and from using fallback policy (FB). To further detail the proportion of actions in the NatBDI intention

Table 4.3 – ScienceWorld results from NatBDI execution using two different fallback policy types: Swift and DRRN. We report the results by detailing the total scores, the scores from plans (BDI), and those from fallback actions (FB), as well as the average number of actions executed from plans (BDI) and from the fallback policy (FB). We highlight which approach that contributes more in the total score in bold.

Task	**Num Plans**	**Swift**			**DRRN**		
		Score (Total)	**Score (BDI/FB)**	**Actions (BDI/FB)**	**Score (Total)**	**Score (BDI/FB)**	**Actions (BDI/FB)**
find-non-living-thing	0	0.91	0.00/**0.91**	0.00/10.43	0.66	0.00/**0.66**	0.00/50.00
	8	0.92	0.32/**0.60**	3.33/6.91	0.77	0.32/**0.45**	3.33/34.68
	15	0.92	**0.60**/0.32	6.25/5.28	0.86	**0.60**/0.26	6.25/20.40
	23	0.97	**0.81**/0.16	7.64/2.68	0.93	**0.81**/0.12	7.64/9.52
	30	1.00	**1.00**/0.00	9.19/0.00	1.00	**1.00**/0.00	9.19/0.00
melt	0	0.21	0.00/**0.21**	0.00/50.00	0.03	0.00/**0.03**	0.00/50.00
	4	0.32	0.11/**0.21**	5.00/45.33	0.14	**0.11**/0.03	4.78/45.33
	7	0.54	**0.34**/0.20	10.11/34.00	0.36	**0.34**/0.02	10.33/34.00
	10	0.60	**0.56**/0.04	15.89/22.67	0.57	**0.56**/0.01	15.89/22.67
	13	0.79	**0.78**/0.01	20.33/17.00	0.78	**0.78**/0.00	20.56/17.00

structure, we report the average number of actions (Actions) originated from plans (BDI) and from the fallback policy (FB). The number of episodes seen in DRRN training for “find-non-living-thing” and “melt” are 302 and 457 respectively.

In “find-non-living-thing” task, the Swift fallback policy achieves a high score even in scenarios with no plans (i.e., setup with 0 plan-rules), surpassing the DRRN approach by a large margin. Notice that Swift employs a significantly larger neural network (i.e., 770 million parameters) than the DRRN policy network (i.e., 1.5 million parameters). Recall from Section 4.2 the “find-non-living-thing” task is the easiest task from the ScienceWorld environment, and the resulting scores from both approaches are noticeably higher compared to the “melt” task. In this task, as the setup that uses Swift fallback policy already achieves better results without plans than DRRN, it requires a larger number of plans to improve the score. In contrast, the agent configuration that uses DRRN as fallback policy heavily relies on natural language plans to improve its score in the “find-non-living-thing” task, highlighting the difference between the performance of both approaches.

Regarding the “melt” task, although Swift obtains better scores than DRRN, both fallback policy approaches result in poor performance in a task that requires more sophisticated reasoning capabilities. By including a few extra plans (e.g., 4 and 7 plans for the “melt” task), only the Swift fallback policy contributes to the overall score. Such examples corroborate that the fallback policy can partially handle situations where the agent does not have a plan to solve the goal, considering the generalization capabilities of the fallback policy grounding model. Specifically, in such scenarios where the developed plan does not have enough steps to conclude the task objective, the Swift fallback is able to generate a new plan on the fly by predicting a sequence of actions to progress towards the

main goal. However, in the same scenario, the DRRN policy could not effectively proceed with the agent's execution towards the goal after failing to find any plans, as it could not contribute to the overall score. Recall from Section 4.3 that we develop plans to cover 7 out of 9 "melt" test variations, which explains the 0.78 score that NatBDI obtains in the last row of Table 4.3.

Our experiment shows that DRRN policy by itself could not deal with important aspects of the environment, such state-of-matter and target objects, or reason over the task in both tasks, which corroborates the results presented in [80]. Particularly, in the "melt" task, the DRRN-only agent (i.e., the agent that has no user-supplied plan rules) obtains a score greater than zero due to random move actions that send the agent to the exact location of the target element. Although the trained policy could achieve better scores in the "find-non-living-thing", its performance is due to the environment which rewards the agent with a 0.5 score by solely picking a non-living-object, which is more predominantly featured in the environment than living objects. The DRRN fallback policy often correctly picks up a non-living object as mentioned earlier, but struggles to move it to the container of the task description, which lowers the overall score. Even a small number of plan-rules leads to the BDI agent outperforming the DRRN-only by a large margin in the hardest task (melt).

Recall from Sections 2.4.4 and 2.4.5 that a DRRN uses only the text that the environment provides to the agent in the current turn (i.e., observations, inventory, scene details), whereas Swift uses a prompt containing substantially more information about the context such as previous actions and observations, rooms visited, etc.Besides the fact that Swift uses a larger model, the substantial performance difference between both approaches shows the difference of using a large context to describe the input. That is, the Swift approach is able to interpret larger input texts than DRRN, allowing the fallback policy to incorporate more information into the input from the belief base to predict the action. Training policies with additional information by representing textual perceptions through larger input texts enables the agent to retain more details about the environment [11].Using a large model such as T5 [54] enables Swift to capture more sophisticated patterns within a large input text during the training phase, thereby providing a significant advantage compared to DRRN.

Introducing natural language plan-rules also leads to the agent requiring fewer actions (i.e., using shorter plans) to achieve a goal as a result of our natural language plan-rules encoding nearly optimal trajectories in their plan body to solve the task. Notice that configurations that use fewer plan rules rely more on the fallback policy and, consequently, lead the agent to generate more fallback actions. Except for the configuration that uses the Swift module (e.g., Line 1 of Table 4.3, in task "find-non-living-thing"), all fallback policy executions that do not use human-provided plan-rules fail to achieve the

task description objective, as the number of fallback actions reaches the limit (50) along with the score lower than 1.

The BDI agent achieves better scores by using manually designed natural language plan-rules, but it does not obtain a perfect score since we use a limited number of plan-rules, which do not cover all possible variations of the tasks. This is a more realistic setting, as we would not expect the developer to be able to create all possible plan-rules. For example, in the task “melt”, there are variations in which some objects are not working (i.e., broken stove), which requires the agent to plan more dynamically to find an alternative plan at runtime.

4.5.2 Effects of Including Plans And Training Episodes

In this section, we examine the effects of increasing the number of episodes throughout the DRRN training process compared to incorporating more natural language plans. Recall from Table 4.3 that the DRRN-based fallback policy fails to improve the NatBDI reasoning capabilities in the ScienceWorld tasks as its resulting score does not improve the agent’s overall score. Thus, we analyse further the training process of the DRRN network by generating a set of different checkpoints, distinguishing them by the number of trained episodes to evaluate the training efficiency and compare it within NatBDI as a fallback policy component.

While the policy efficiency improves throughout the training phase for the “find-non-living-thing” task, it is prone to overfitting. Recall from Section 4.2 that the task objectives seen in the Scienceworld test variations involve different substances compared to those in the training variations. Training on more episodes from training variations causes the policy to overfit to the training objectives. For example, adding too many episodes that handle the objective “melt water” misleads the policy when handling “melt gallium”, as it overfits to trajectories focused on the water melting point rather than that of gallium. Increasing the number of episodes seen during training enhances the DRRN score until it reaches a saturation point after using 302 episodes, which represents the best configuration that we use in Table 4.3. However, increasing the number of plans yields substantially greater improvements for the agent than prolonging the training process with additional episodes. In fact, extending the number of episodes causes the DRRN parameters to overfit during training and consequently degrades performance in the test variations of the task “find-non-living-thing” after surpassing 535 episodes, as Figure 4.5 indicates.

In the “melt” task, the number of training episodes does not seem to affect the performance of the DRRN. This makes sense since we have seen in previous results that DRRN performs very poorly in this task, and therefore increasing the number of training

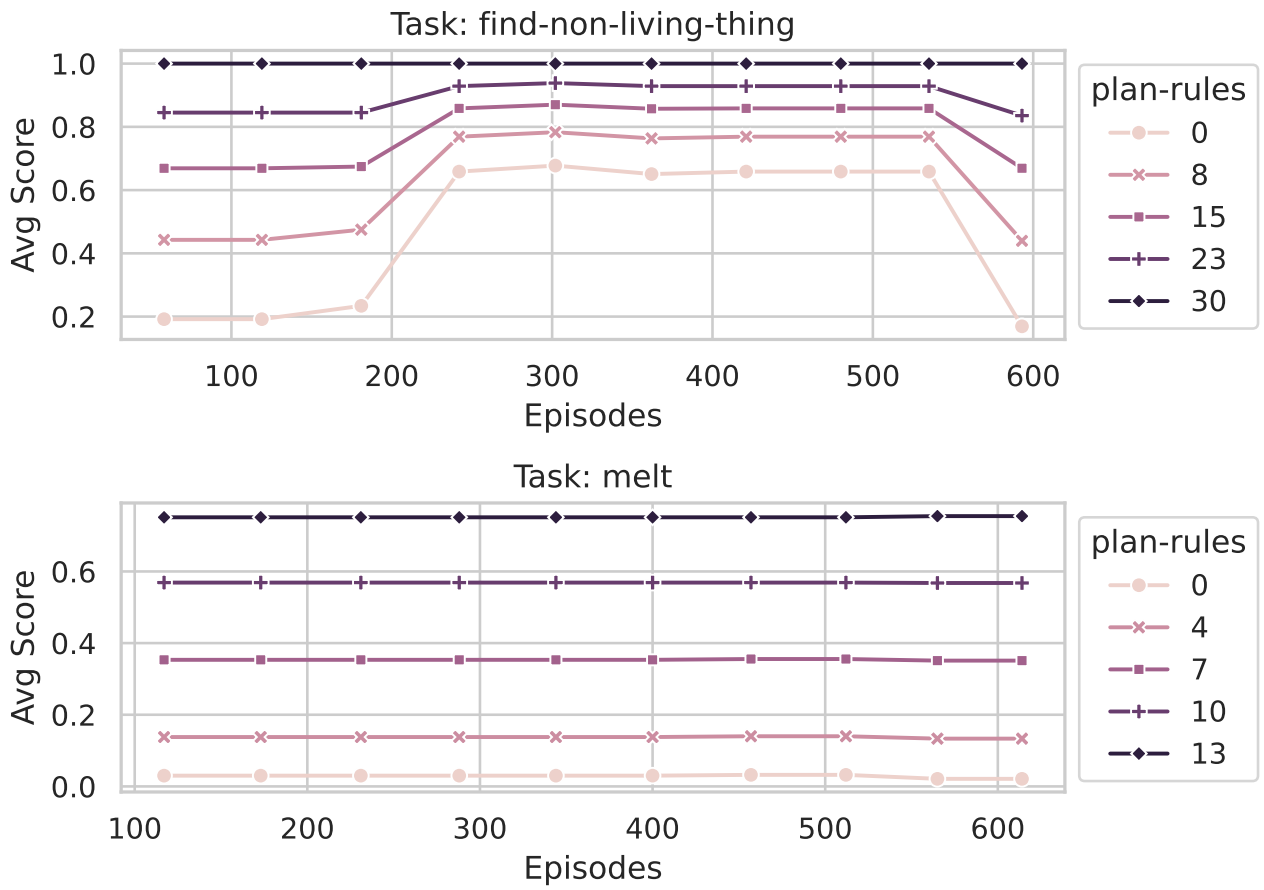


Figure 4.5 – Scores per episodes when scaling the number of plan-rules and the number of training episodes.

episodes here has no discernible effect. This is evident in the task “melt” curve in Figure 4.5 as it does not change as the number of episodes increases. Thus, this corroborates that DRRN has major limitations in reasoning over sophisticated tasks (i.e., “melt”) that require planning by interpreting natural language states.

4.5.3 Natural Language Inference Analysis

As a final set of experiments, we report the impact of using different language models as the NLI model in the NatBDI reasoning cycle. We evaluate three language models with a distinct number of parameters fine-tuned on the MultiNLI dataset (MNLI) to analyse their efficacy in the ScienceWorld “melt” task. In this section, we structure this NLI model experiment as follows: First, we detail the scores obtained in each ScienceWorld task for each NLI model and the effects caused by misleading inferences. Second, we discuss the use of sentences processed using lexical overlap to measure the contrast between beliefs and plan context sentences.

Effects in Plan Selection.

In this experiment, we focus our analysis on the number of missed plans caused by incorrect inferences in the NatBDI reasoning cycle. For each NLI model, we report the scores along with the average number of actions from plans, and metrics to illustrate the quantity of missed plans in Table 4.4. Additionally, we include the accuracy obtained on MNLI-m [83] dataset to compare the score from the "melt" task of each model against an established external benchmark.

NatBDI executes plans hierarchically, following the traditional reasoning cycle of BDI agents. Consequently, errors encountered during plan decomposition will abort the plan-selection process, making later subgoals unreachable. Incorrect inferences between beliefs and plan contexts lead to incorrect plan selection, causing the agent to select irrelevant or unachievable plans or stop its execution early due to not finding any candidate plan. Thus, we report the score in the ScienceWorld "melt" task detailing the number of missed plans due to incorrect logical inferences performed by NLI models in Table 4.4.

Table 4.4 – Experimental results of logical inferences from using different NLI models in NatBDI across all "melt" task variation. We show the best model, the smaller model and the best results in bold. The following columns describe the model name, number of parameters (in millions), the MNLI accuracy on dev set, the score, the average number of actions performed, total plan executions, total missed plan executions and its percentage.

Model	**Params**	**MNLI-m**	**Score (BDI)**	**Num Actions (avg)**	**Plans (Total)**	**Missed Plans (Total)**	**Missed Plans (%)**
MiniLM L6	**22**	82.20	0.39	16.56	41	17	0.41
Bert Base	110	84.60	0.56	16.78	46	12	0.26
Roberta Large	355	**90.80**	**0.78**	**22.11**	**58**	**0**	**0.00**

Recall from our previous experiments (Sections 4.5.1 and 4.5.2) that we use the *roberta-large* [39] NLI model as it successfully executes all of our manually developed plans, resulting in zero missed plans this NLI model. In fact, the *roberta-large* NLI module results in scores that are higher than smaller language models such as *bert-base* and *MiniLM*, in both the MultiNLI matched test set and in the ScienceWorld "melt" task. Part of this experiment is to measure to what extent the natural language inference model influences planning success. We would expect that large models improve the plan selection and, consequently, the plan's success. Table 4.4 shows that using the *roberta-large* model for natural language inference leads to the selection of more plans during the agent's execution, resulting in a superior score compared to the other models. The number of missed plans when using smaller models, such as *bert-base* [20] and *MiniLM* [81], explains the lower scores as the agent is unable to execute all steps of the expected plans due to incorrect inferences. The lower number of actions from plans from such models highlights the incomplete execution of the developed plans across all task variations.

Although including a smaller model such as *MiniLM* leads to a more computationally efficient execution of NatBDI, incorporating it in NatBDI leads to 41% of missed plans and a significant score decreasing due to not executing plans as expected. This indicates that larger models lead to more successful plan selection when used as the NLI model in the NatBDI architecture [8]. Thus, we show that incorporating NLI models using a more robust approach improves the intention progression of the NatBDI by selecting plans that align with human instructions.

Notice that the number of total plan executions in Table 4.4 counts the unique executions across all test variations of the “melt” task, which may include repeated plans. That is, it counts the number of triggered plans that NatBDI performs during execution considering all task variations. In contrast, the number of plans from table 4.3 represents the number of distinct plans included in the plan library.

Lexical Analysis.

This experiment consists of a lexical analysis of sentence pairs comprising beliefs and statements from the plan context that we used in the previous experiment. We report lexical metrics to measure the complexity of detecting the logical relation between sentences that the NLI models process. To analyse the difficulty of inferring entailment between the belief base and the plan context, we compute the lexical overlap between sentence pairs along with the average number of words in both belief and plan context sentences.

We compute the lexical overlap between sentences used as premise and hypothesis to count the number of identical words between the pair. Given a sentence pair consisting of a belief and a context, we compute the number of words contained in beliefs that are absent in the plan context. In cases where lexical overlap is high between the premise and the hypothesis, the inference mechanism tends to more easily infer entailment relation since both sentences are similar and may express the same idea [33, 57]. For example, it is trivial for an NLI model to infer entailment between the belief-context pair “you see a pot” and “you see a container” due to the large lexical overlap (i.e., 3 words). Hence, in such cases, sophisticated language models exploit shallow syntactic heuristics to infer logical entailment between sentences [42, 57].

In Table 4.5, for each evaluated NLI model, we provide the following information: First, we report the average lexical overlap contained in sentence pairs to describe the average number of words shared between belief and plan context. Second, we present the number of sentence pairs classified as entailment to summarize the number of pairs evaluated in this analysis. Finally, we report the average number of words in sentences that represent the belief and the plan context sentence, respectively.

Table 4.5 – Experimental results of our lexical analysis on all entailment sentence pairs. We report the average lexical overlap, the number of pairs, and the average word count in beliefs and plan contexts.

Model	**Lexical Overlap (avg)**	**Num Pairs (Entailment)**	**Belief Num Words (avg)**	**Plan Context Num Words (avg)**
MiniLM L6	1.35	26	8.77	5.12
Bert Base	1.25	28	9.18	5.25
Roberta Large	1.28	32	8.44	5.50

The amount of lexical overlap is low when compared to the average word number in both sentences considering our manually designed plan-rules. As the NLI model progresses through the NatBDI reasoning cycle execution by selecting more plans, the number of sentences pair inferred as entailment also increases. In our experiment, the lexical overlap is significantly low when compared to the average number of words in each belief-plan context pair. For example, in *roberta-large* inferences, the sentence pair inferences have an average of 1.28 common words, while the belief and plan context sentences have averages of 8.44 and 5.50 words, respectively. Thus, we demonstrate that NatBDI NLI implementation is robust to the linguistic variability present in natural language sentences as it could detect entailment relation in sentences expressed in different ways.

5. DISCUSSION

In this thesis, we developed a novel BDI approach to handle tasks in environments written in natural language. In the NatBDI architecture, the agent's mental state consists entirely of natural language representation, making its behaviour transparent and comprehensible to humans. Traditional BDI agents [9, 1, 65] model the reasoning process by using a first-order logic language to apply more formal methods. In contrast, we apply the BDI reasoning process directly to the natural language information, focusing on a more informal reasoning approach by employing natural language inference (recall from Section 2.1.2), where beliefs, goals and plans are expressed as textual sentences.

To support logical inferences in natural language, we introduce a new approach for developing BDI plans inspired by the AgentSpeak semantics [55]. NatBDI's combination of beliefs written as natural language sentences and a planning language provides the first steps towards allowing humans with virtually no programming skills to instruct the agent using natural language. Using logic languages allows the use of lifted variables and grounded predicates, which introduces a more expressible language to develop agents despite requiring a basic programming level from humans. Besides being easier to understand, we adopt a controllable natural language for developing plans to represent all information within the NatBDI reasoning process in natural language. Natural Language Inference models enable inferences based on monotonic relations between terms (e.g., "you see a pot", "you see a container") since annotated datasets [3, 83, 57] explore this linguistic relation in their labelled sentence pairs. However, to fully support lifted variables into the plan language, we still need to include a more sophisticated lexical tool to identify unified terms (e.g., container $\mapsto$ metal pot) to perform substitution over the plan body steps [43]. Thus, in this thesis, NatBDI performs logical inferences by considering sentence words as grounded terms. In future work, we plan to address language operations, such as substitution and unification, to support lifted variables without neglecting the natural language aspects.

The NatBDI interpreter triggers plans by analysing the sentence that represents their invocation condition with goal addition events from the NatBDI's event queue. In contrast to traditional BDI implementations [1, 55], which support triggering plans through belief addition events, NatBDI triggers plans exclusively based on goal additions. In text-based environments [28, 80, 15], the agent perceives multiple beliefs simultaneously, requiring a more sophisticated method to manage parallelism in NatBDI plan selection mechanism. For example, in Figure 3.7, the agent perceives the beliefs "you see a cupboard" and "you see an open door to hallway" at the same time, which might trigger two concurrent plans due to belief additions.

Recent works that employ machine learning use large volumes of data to develop agents for handling text-based environment tasks more autonomously, although they of-

ten rely on opaque models [89, 38]. That is, such models lack a transparent reasoning process, which makes it difficult for humans to understand the intrinsic motivations behind the agent's behaviour [35]. As we mentioned before, supplying instructions for the agent allows human to easily control its behaviour, though it requires more human effort. To develop an autonomous agent that is more controllable while reducing human effort, we introduce a component to predict an action when no candidate options are available to the agent. Since the focus of this thesis is to develop a new BDI architecture, we choose to use and adapt existing machine learning approaches [29, 38] by seamlessly integrating them into the NatBDI reasoning cycle instead of creating a new one.

In this thesis, we conduct experiments in the ScienceWorld environment to evaluate the efficiency of the NatBDI implementation by using plans and fallback policy actions. Although we conduct our experiments only in the ScienceWorld environment, this thesis does not limit the NatBDI architecture to it, as long as the text-based environment models a POMDP by following concepts that we detail in Section 2.3.3. Given a set of plan library configurations with different numbers of plans, our experiments show that the machine learning approaches that we evaluate as fallback policy still face challenges in the ScienceWorld environment. Although it requires manual human effort, we demonstrate that including manually designed plan-rules can improve efficiency in ScienceWorld tasks. We could not evaluate the most recent large language models due to the computational resources required for running them. As future work, we plan to incorporate other machine learning model alternatives grounded by more recent large language models [11].

Recall from Section 3.1 that we highlight important concepts for introducing autonomous agents to address tasks in real-world such as transparency, controllability, and explainability [78, 85, 86, 60]. Below, we establish a relation between the concepts from Verhagen et al. [78] and the NatBDI architecture discussing the following points:

- Transparency: We argue that introducing a natural language belief base allows the agent to provide information about its internal state. Beliefs represented by natural language sentences ease the validation process for humans to scrutinize their behaviour, disclosing information about the agent's internal state. As we detail in Figure 3.7, the NatBDI architecture clearly describes the logical inferences performed, which reveal beliefs that entail each statement and enable humans to recognize the motivation of each selected plan. In this example, the agent discloses that the agent believes it has a metal pot to handle gallium, and it is in the kitchen to open the door to the hallway to move towards the workshop to get it.

- Controllability: NatBDI's combination of beliefs written as natural language sentences and a planning language provides the first steps towards allowing humans with virtually no programming skills to instruct the agent using natural language. Recall from Section 4.5 that using a pure machine learning approach (i.e., 0 plans),

the agent faces limitations regarding the complexity of reasoning in a text-based environment. In our experiments (Section 4.5.1), we demonstrate that humans can intervene in the agent's behaviour using natural language plans to guide it toward achieving the goal. In the example from Figure 3.7, the plan library contains plans that instruct how to obtain gallium in the ScienceWorld environment, which informs to get a metal pot for manipulating the substance. Even though the fallback policy encapsulates opaque models to predict actions, recall from Algorithm 3.3 that the agent still prioritizes plans from plan library, which enables the human developer to overrule the fallback policy by using a specific plan when necessary. That is, the fallback policy will include actions in the intention structure only when no candidate plan is available.

- Explainability: The NatBDI provides the agent's internal information by disclosing its belief base in a human-understandable representation, enabling humans to identify the causes of its decisions, goals or beliefs. We argue that a natural language belief base supports further behaviour analysis to find belief-based explanations behind each agent's decision. Given the example illustrated in Figure 3.7, consider the case of the "get gallium" plan adoption. By providing a belief-based explanation, the agent can clarify that it adopts such a plan because it perceives that the metal pot is already in its inventory. In the context of this thesis, we follow the Verhagen *et al.* approach to refer to explanations as a more informal verification process, which aims to analyse internal information of the agent to identify the causes of its behaviour. As future work, we plan to employ BDI formal methods to analyse the NatBDI natural language components [45, 19].

6. RELATED WORK

In this chapter, we briefly explain work related to our thesis research and compare them with our approach, organizing them according to their contribution as follows. First, we survey works that employ BDI agents in natural language environments and compare them with our approach. Second, we discuss works that integrate machine learning techniques in the BDI architecture. Finally, we describe works that introduce approaches to perform logical inferences over natural language information.

6.1 Traditional BDI Agents in Natural Language Environments

We start with work that employs BDI agents to handle natural language processing tasks. Most of the related work that applies BDI in textual domains cover conversational agents and personal assistants, requiring understanding and communicating with humans. Although our work does not focus on such problems, we compare methodologies that model BDI agents to perceive and act over natural language.

The approach from Nestorovic et al.[48] introduces a BDI model for task-oriented dialogues focusing on information representation and deliberation process. This work formulates the agent's beliefs by extracting the user intention (i.e., dialogue act) and data (i.e., slots) and separating them into different layers to represent the dialogue context. Given this belief formulation schema, they apply a set of rules and heuristics over the representation of the dialogue context to infer the agent response. Specifically, to maintain the dialogue with a user, the agent requests more information to maintain the user talking by using predefined dialogue heuristics based on strategies grounded in psychology. Although this work handles task-oriented dialogues similar to ours, our work diverges in the dialogue progress since we train a dialogue policy and leverage an AgentSpeak-like formalism to introduce developed plans instead of relying on heuristics to proceed with the dialogue with a human.

Wong et al. [88] introduce a goal-oriented architecture based on BDI to develop a conversational virtual companion aiming to entertain children and do collaborative tasks. This architecture consists of a component to manage the content of agent response, named Activity Management, and a component that interprets the perceived utterances and maps them to strategies defined as a BDI plan, named Conversation Management. Similar to the work above, this approach extracts keywords and aspects such as sentiment and entities mentioned from input utterances and identifies whether the information extracted matches a pattern predefined by a human.

Nguyen and Wobcke [49, 50] develop an approach consisting of a Smart Personal Assistant (SPA) by extending the BDI agent architecture. Their agent implementation focuses on tasks involving email management support, which requires the SPA to interpret the user intention and generate a response. Such SPA consists of a dialogue manager that uses the BDI reasoning cycle to formulate the SPA response through developed plans.The agent architecture contains a rule-based parser to represent input utterances in a logical representation similar to traditional BDI approaches.

Both approaches handle natural language information by employing semantic parsers grounded in predefined rules or linguistic syntax properties to convert text into a formal representation. Instead of applying reasoning over an intermediate formal representation, our work focuses on developing the agent's reasoning cycle that processes natural language information directly by integrating existing machine learning methods. That is, we model a BDI agent by encoding its mental components entirely in natural language to handle tasks in a text-based environment using natural language plans.

6.2 Machine Learning Techniques and BDI Agents

We now focus on work that integrates machine learning approaches into the BDI architecture, and contrast the differences in terms of research objectives and methodologies. Our work aims to integrate machine learning approaches to handle situations where there is no applicable plan, given the current representation of the agent's belief base (Section 3.5). Traditional BDI agents do not incorporate adaptative mechanisms to improve their decision-making capabilities during its execution beyond predefined plan rules in their plan library. However, following predefined plan rules ensures that the agent behaves according to human instructions.

Singh et al. [66, 67] develop an approach to determine plan conditions (i.e., plan context) by employing machine learning techniques in environments with changing dynamics to improve hierarchical plan selection in the JACK agent programming language [9]. That is, this work introduces a mechanism that adapts the plan-rule context by adjusting to changes in the environment in order to use the same instructions encoded in existing plan-rules. Concretely, this work uses a decision tree that leverages previous experiences of plan executions in different situations, as it can represent context formulas in the same way BDI languages such as JACK encodes the plan context formula. In our research, we apply machine learning to handle text-based environment tasks in cases where there is no plan available instead of adapting the existing ones in plan library. Although this work has a different objective in applying machine learning to BDI agents compared to ours, we envision integrating a similar approach to infer whether the agent can select an existing plan in future work.

Later, Singh and Hindriks [65] work introduces an agent programming language, named GOAL, that aims to improve action selection by applying machine learning techniques. This work employs the q-learning algorithm to obtain a policy that selects the candidate action with the highest q-value in cases where multiple options are available. Similar to the existing models we encapsulate in the fallback function (recall from Section 4.4), Their approach encodes state information using the belief base and goals to compute the q-values for actions, which the agents develop using the GOAL language. This work shares the same objective as our approach to developing the fallback policy using machine learning, but there are conceptual differences as follows: First, we include only the main goal which the environment provides as input to the fallback policy. Second, our work encodes both plans and the agent's mental model into a natural language representation, which poses more challenges for performing logical inferences compared to symbolic representations.

Bosello and Ricci [2] integrate a traditional BDI agent framework with a reinforcement learning approach to improve the agent developing process using Jason programming language [1]. Concretely, this approach uses reinforcement learning to learn a policy function to select actions at runtime and at the same time reduce the necessity of creating new plans manually. By introducing learning into the agent's development process, Bosello and Ricci introduce two different types of plans: hard plans, which consist of manually developed plans as in the traditional way; and soft plans, which consist of actions generated by the learned policy function. This work applies machine learning techniques in the same interpreter stage as our work (i.e., plan selection), where hard plans correspond to NatBDI developed natural language plans (recall Section 3.4.1), and soft plans correspond to the NatBDI fallback policy actions. As we compare to other BDI related works above, the main contrast with our work is that NatBDI employs reinforcement learning over textual information, which processes both agent's information (e.g., beliefs, goals) and plans written in natural language.

6.3 Logical Inference on Natural Language Information

Finally, we briefly survey methods to handle logic inference over natural language sentences. First, we detail traditional approaches that handle more formal approaches to develop natural logic systems for inference in language. Second, we discuss more recent approaches that employ modern neural network models to make logical inferences by learning linguistic patterns in logic annotated datasets.

6.3.1 Natural Logic Methods

Valencia's work [74] develops a Natural Logic model based on a mechanism that applies monotonicity calculus. In the Natural Logic context, monotonicity calculus describes entailment as semantic containment relation between natural language texts, which is analogous to the set containment relation. Therefore, this analogy allows classifying bi-directional relation distinguishing forward entailment and reverse entailment. For example, the word pair ("crow", "bird") is a forward entailment whereas the pair ("European", "French") is a reverse entailment. However, the approach of monotonicity calculus lacks any representation of semantic exclusion such as negation and alternation. Thus, using many simple inferences fails using monotonicity calculus such as "Stimpy is a cat" $\models$ "Stimpy is not a poodle". Assuming that the word pair "poodle" and "dog" is a forward entailment, Stimpy cannot be both a cat and a dog simultaneously. However, since the hypothesis negates that Stimpy is a poodle, this example represents a valid inference.

MacCartney *et al.* [41] develop an extension of Natural Logic to incorporate semantic exclusion in monotonicity calculus. Following Valencia, MacCartney *et al.* work formalizes an inventory of relations that represents semantic containment including two new relations to represent semantic exclusion: negation and alternation. Negation is analogous to set complement while alternation is analogous to exclusive disjunction. MacCartney *et al* [41] uses the $\wedge$ symbol to evoke the logically similar bitwise XOR operator of the C programming language family, although it is the same symbol used to represent logical conjunction. This extension includes a cover relation to represent non-exhaustive exclusion (non-equivalence). Table 6.1 describes the symbol defined by MacCartney *et al.* [41] that represents each relation with an example.

Table 6.1 – Inventory of semantic relations of MacCartney *et al.* [41] extension of Natural Logic

Symbol	Name	Example
$x \equiv y$	equivalence	sofa $\equiv$ couch
$x \sqsubset y$	forward entailment	crow $\sqsubset$ bird
$x \sqsupset y$	reverse entailment	European $\sqsupset$ French
$x \wedge y$	negation	human $\wedge$ nonhuman
$x \mid y$	cover	cat $\mid$ dog
$x \# y$	independence	hungry # hippo

Both natural logic studies have an essential role in more recent works in natural language inference, providing a framework for developing natural logic systems. In this work, we focus on applying machine learning techniques that follow natural logic concepts to make inferences in natural language. That is, we employ a model that performs logical

inferences focusing on a more informal form of reasoning that covers linguistic aspects such as ambiguity and the variability of linguistic expressions [40].

6.3.2 Logical Inference In Language Applying Machine Learning

Williams *et al.* [83] presents a natural language inference (NLI) dataset to train machine learning approaches to infer the logical relation between sentences, named Multi-Genre Natural Language Inference (MNLI). The MNLI corpus is a large dataset that contains 433k sentence pairs annotated with NLI classes (*contradiction, entailment, neutral*) [83]. The MNLI dataset contains diverse sentences including text and speech, covering a wide range of topics. In this work, we conduct experiments using different NLI models [39, 81, 20] trained using the MNLI train dataset. However, we merge the classes *neutral* and *contradiction* into a single class *non-entailment* since our focus is on identifying whether there is an entailment between sentences.

Richardson *et al.* [57] work introduces an NLI dataset that aims to evaluate different types of reasoning capabilities of machine learning approaches. Specifically, Richardson *et al.* organize the dataset into different semantic phenomenon, each one represents the following logic fragments: boolean coordination, quantification, conditionals, comparatives, and monotonicity reasoning. They train a BERT [20] language model on their dataset and demonstrate that BERT can learn logic fragments as long as the training dataset contains sentence pairs with the corresponding fragment type. That is, given a new semantic fragment, the model must be retrained with data containing sentence pairs that reflect that fragment. In future work, although the authors did not provide the trained models, we intend to integrate language models trained with such logic fragments to evaluate the NatBDI plan selection mechanism.

Clark *et al.* [14] work investigates the capabilities of transformer-based neural networks to reason over facts and rules using synthetic generated data. Given a set of sentences representing facts and a hypothesis sentence, the model should determine whether the facts follow the hypothesis by performing deductive inference over English statements. They show that transformers can reason with facts expressed in natural language as long as the sentences consist of a predefined format (e.g. “A is B”, “if A and B then C”). In our work, we prefer to use models that follow the natural language inference task (recall Section 2.1.2) that handles sentence pairs to perform logic operations more equivalent to the traditional BDI methods.

7. CONCLUSION

In this thesis, we introduce a novel autonomous agent architecture based on BDI concepts to solve tasks in environments that require human language comprehension to make decisions over a natural language action space. We develop the NatBDI architecture, which represents the BDI mental components entirely in natural language that allows humans to understand the information behind the agent's behaviour. In contrast to traditional BDI approaches, which encode their internal state in a symbolic representation to leverage formal logic systems, NatBDI belief base comprises beliefs encoded as descriptive sentences detailing what the agent is seeing. The NatBDI architecture presents an alternative for developing autonomous agents that consist of more transparent components and more controllable behaviours when compared to agents purely based on machine learning models. Combining beliefs written as natural language sentences and a planning language provides the first steps towards allowing humans with virtually no programming skills to instruct the agent using natural language.

We organize the conclusion chapter of this thesis as follows: In Section 7.1, we enumerate and summarize the main contributions of this thesis. In Sections 7.2 and 7.3, we detail, respectively, the open issues and limitations of our thesis and future work that we intend to focus on to address such scenarios.

7.1 Contributions

The main contribution of this thesis is an agent architecture that bridges the gap between the BDI models and real-world tasks encoded in natural language.We summarize the main contributions of this thesis to the BDI and the natural language processing research areas as follows:

- We develop a new class of agent architecture (NatBDI) by introducing the BDI practical reasoning method over natural language environments 3.2. NatBDI represents all of its BDI components (i.e., beliefs, desires, and intentions) as natural language sentences, with a more transparent mental state to humans.

- A novel method to select plans written in natural language by using natural language inference models (Section 3.4.2). NatBDI employs a neural network pretrained on a dataset annotated with logical relations to infer whether a plan context is a logical consequence of the agent's belief base.

- A new approach to develop BDI plans written in natural language, enabling humans with no programming skills to instruct and control autonomous agents (Section

3.4.1). The plan language uses a semantic inspired by the AgentSpeak language [55] to represent triggering events, plan context and the plan body as sentences.

- A new intention-failure mechanism to handle scenarios where no candidate plans is available, relying on a fallback policy trained by reinforcement learning approaches (Section 3.5). We adopt existing reinforcement learning methods by encapsulating them into a function that maps the belief base to an executable action.

7.2 Open Issues and Limitations

The contributions of this thesis are centred on modelling BDI agents for addressing tasks in text-based environment using a natural language mental state. In this section, we discuss open issues and limitations of NatBDI architecture to develop agents to address text-based environments, and identify alternatives to overcome such points in future work.

As NatBDI processes natural language texts without relying on a pre-determined format, the reasoning cycle does not manipulate lifted variables as traditional BDI agents do, assuming that every term is a grounded variable. That is, the natural language inference model does not handle substitutions and does not identify unification relations while performing logical inferences in the plan context or executing intentions. Identifying when an expression represents a lifted or grounded is a challenging task that arises when manipulating natural language expressions. A possible path to develop a method to handle lifted variables is to explore relations from monotonicity calculus [74] by identifying forward entailment or other semantic relations between sentence words.

Simulated text-based environments [80, 15, 28] require the agent to perform an action in order to cause changes in the environment, considering only synchronous events. Consequently, NatBDI can only select and execute plans by reacting to events resulting from its own actions. However, in a simulated environment that more closely reflects real-world conditions, the agent should be able to perceive events arising from external situations and reason about new perceptions not necessarily generated by its own actions. An alternative to overcome this limitation is to improve the NatBDI to handle belief addition/deletion events [56] similar to existing BDI agents, which handle asynchronous events by triggering plans or actions based on new perceptions.

In order to find candidate options for the agent to perform, NatBDI inference method requires scanning every belief-plan context sentence pair in order to conclude the entailment relation. Bosello and Ricci [2] developed a method to extract observations from the belief base according to the goal to reduce the state space on top of Jason. Matching the natural language semantics is more complex than handling formal languages since a belief can express the same meaning using different expressions. As an alternative,

we plan to explore methods to measure semantic similarity [33, 23] between plan context statements and beliefs to filter irrelevant pairs and, hence, reducing the state space similar to the observation function approach from Bosello and Ricci work.

7.3 Future Work

Developing a BDI architecture with mental states that use a natural language representation opens multiple avenues for creating more efficient and trustworthy autonomous agents to address real-world problems. With respect to our approach to perform logical inferences in natural language statements, we intend to extend this work as follows.

First, we intend to improve the operational semantics of the NatBDI interpreter to increase robustness in the natural language plans as well-known formalisms in BDI do [84, 55]. Concretely, we intend to include support for lifted expressions, and disjunctions in the natural language plan contexts. Given natural language plans having lifted terms in their plan context and plan body steps, the reasoning cycle should perform substitutions analogously to traditional BDI approaches in first-order logic languages. Using lifted terms in natural language plans enables humans to create more generalizable plans for NatBDI, while disjunctions between natural language statements allow for the development of more complex plan contexts in the NatBDI plan library.

Second, we intend to explore more recent LLMs to apply inferences without relying on annotated dataset or a specialized model (i.e., zero-shot classifiers [68, 27]). Besides requiring large and expensive computational resources, LLMs handle a wide-range of natural language tasks without the need for a task-specific annotated dataset by interpreting the user inputs (i.e., prompts) and generating comprehensible responses. A possible alternative is to incorporate the premise and hypothesis sentences into an input prompt and use the LLM's response as the logical inference result.

Finally, we plan to explore is the development of an automatic approach for generating natural language plans. Creating natural language plans still requires manual human effort to introduce agents into text-based environment. We intend to develop a method to automatically create plans by recognizing possible goals for the agent to achieve, inferring the plan context and identifying the steps to incorporate in the plan body. Generating natural language plans in a more autonomous manner reduces manual efforts in developing plans and maintains the premise of designing transparent, controllable and explainable agents.

REFERENCES

[1] Bordini, R. H.; Wooldridge, M.; Hübner, J. F. “Programming Multi-Agent Systems in AgentSpeak using Jason”. Chichester, UK: John Wiley & Sons, 2007, 304p.

[2] Bosello, M.; Ricci, A. “From programming agents to educating agents–a jason-based framework for integrating learning in the development of cognitive agents”. In: International Workshop on Engineering Multi-Agent Systems, 2019, pp. 175–194.

[3] Bowman, S. R.; Angeli, G.; Potts, C.; Manning, C. D. “A large annotated corpus for learning natural language inference”. In: Proceedings of the Conference on Empirical Methods in Natural Language Processing, 2015, pp. 632–642.

[4] Brafman, R. I.; Tennenholtz, M. “R-MAX - A general polynomial time algorithm for near-optimal reinforcement learning”, *Journal of Machine Learning Research*, vol. 3, oct 2002, pp. 213–231.

[5] Bratman, M. E. “Intention, Plans and Practical Reason”. Harvard University Press, 1987, 208p.

[6] Bratman, M. E.; Israel, D. J.; Pollack, M. E. “Plans and resource-bounded practical reasoning”, *Computational Intelligence*, vol. 4–4, sep 1988, pp. 349–355.

[7] Brown, T.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J. D.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al.. “Language models are few-shot learners”. In: Proceedings of the Advances in Neural Information Processing Systems, 2020, pp. 1877–1901.

[8] Brutzkus, A.; Globerson, A. “Why do larger models generalize better? a theoretical perspective via the xor problem”. In: Proceedings of the International Conference on Machine Learning, 2019, pp. 822–830.

[9] Busetta, P.; Rönnquist, R.; Hodgson, A.; Lucas, A. “Jack intelligent agents-components for intelligent agents in java”, *AgentLink News Letter*, vol. 2–1, jan 1999, pp. 2–5.

[10] Cardoso, R. C.; Ferrando, A. “A review of agent-based programming for multi-agent systems”, *Computers*, vol. 10–2, jan 2021, pp. 16.

[11] Chang, Y.; Wang, X.; Wang, J.; Wu, Y.; Yang, L.; Zhu, K.; Chen, H.; Yi, X.; Wang, C.; Wang, Y.; et al.. “A survey on evaluation of large language models”, *ACM transactions on intelligent systems and technology*, vol. 15–3, mar 2024, pp. 1–45.

[12] Chen, L.; Lu, K.; Rajeswaran, A.; Lee, K.; Grover, A.; Laskin, M.; Abbeel, P.; Srinivas, A.; Mordatch, I. “Decision transformer: Reinforcement learning via sequence modelling”.

In: Proceedings of the Advances in Neural Information Processing Systems, 2021, pp. 15084–15097.

[13] Chung, J.; Gulcehre, C.; Cho, K.; Bengio, Y. “Gated feedback recurrent neural networks”. In: Proceedings of the International Conference on Machine Learning, 2015, pp. 2067–2075.

[14] Clark, P.; Tafjord, O.; Richardson, K. “Transformers as soft reasoners over language”. In: Proceedings of the Twenty-Ninth International Joint Conferences on Artificial Intelligence, 2020, pp. 3882–3890.

[15] Côté, M.-A.; Kádár, A.; Yuan, X.; Kybartas, B.; Barnes, T.; Fine, E.; Moore, J.; Hausknecht, M.; El Asri, L.; Adada, M.; et al.. “Textworld: A learning environment for text-based games”. In: Computer Games: 7th Workshop, Held in Conjunction with the 27th International Conference on Artificial Intelligence, 2019, pp. 41–75.

[16] Dagan, I.; Glickman, O.; Magnini, B. “The pascal recognising textual entailment challenge”. In: Proceedings of the International Conference on Machine Learning, 2006, pp. 177–190.

[17] Das, B. C.; Amini, M. H.; Wu, Y. “Security and privacy challenges of large language models: A survey”, *ACM Computing Surveys*, vol. 57–6, feb 2025, pp. 1–39.

[18] de Silva, L.; Meneguzzi, F.; Logan, B. “Bdi agent architectures: A survey”. In: Proceedings of the International Joint Conference on Artificial Intelligence, 2020, pp. 4914–4921.

[19] Dennis, L. A.; Oren, N. “Explaining BDI agent behaviour through dialogue”. In: Proceedings of the International Conference on Autonomous Agents and Multiagent Systems, 2021, pp. 429–437.

[20] Devlin, J.; Chang, M.-W.; Lee, K.; Toutanova, K. “BERT: Pre-training of deep bidirectional transformers for language understanding”. In: Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2019, pp. 4171–4186.

[21] Dziri, N.; Lu, X.; Sclar, M.; Li, X. L.; Jiang, L.; Lin, B. Y.; Welleck, S.; West, P.; Bhagavatula, C.; Bras, R. L.; Hwang, J. D.; Sanyal, S.; Ren, X.; Ettinger, A.; Harchaoui, Z.; Choi, Y. “Faith and fate: Limits of transformers on compositionality”. In: Conference on Neural Information Processing Systems, 2023, pp. 70293–70332.

[22] Fuchs, N. E.; Kaljurand, K.; Schneider, G. “Attempto controlled english meets the challenges of knowledge representation, reasoning, interoperability and user interfaces”. In: Proceedings of the International Florida Artificial Intelligence Research Society Conference, 2006, pp. 664 – 669.

[23] Gao, T.; Yao, X.; Chen, D. “Simcse: Simple contrastive learning of sentence embeddings”. In: Proceedings of the Conference on Empirical Methods in Natural Language Processing, 2021, pp. 6894–6910.

[24] Gendron, G.; Bao, Q.; Witbrock, M.; Dobbie, G. “Large language models are not strong abstract reasoners”. In: Proceedings of the International Joint Conference on Artificial Intelligence, 2024, pp. 6270–6278.

[25] Goecks, V. G.; Gremillion, G. M.; Lawhern, V. J.; Valasek, J.; Waytowich, N. R. “Integrating behavior cloning and reinforcement learning for improved performance in dense and sparse reward environments”. In: Proceedings of the International Conference on Autonomous Agents and Multiagent Systems, 2020, pp. 465–473.

[26] Goodfellow, I.; Bengio, Y.; Courville, A. “Deep Learning”. MIT Press, 2016, 800p.

[27] Gretz, S.; Halfon, A.; Shnayderman, I.; Toledo-Ronen, O.; Spector, A.; Dankin, L.; Katsis, Y.; Arviv, O.; Katz, Y.; Slonim, N.; Ein-Dor, L. “Zero-shot topical text classification with LLMs - an experimental study”. In: Findings of the Association for Computational Linguistics, 2023, pp. 9647–9676.

[28] Hausknecht, M.; Ammanabrolu, P.; Côté, M.-A.; Yuan, X. “Interactive fiction games: A colossal adventure”. In: Proceedings of the AAAI Conference on Artificial Intelligence, 2020, pp. 7903–7910.

[29] He, J.; Chen, J.; He, X.; Gao, J.; Li, L.; Deng, L.; Ostendorf, M. “Deep reinforcement learning with a natural language action space”. In: Proceedings of the Annual Meeting of the Association for Computational Linguistics, 2016, pp. 1621–1630.

[30] Hirschberg, J.; Manning, C. D. “Advances in natural language processing”, *Science*, vol. 349–6245, jul 2015, pp. 261–266.

[31] Ichida, A. Y.; Meneguzzi, F. “Modeling a conversational agent using bdi framework”. In: Proceedings of the ACM/SIGAPP Symposium on Applied Computing, 2023, pp. 856–863.

[32] Ichida, A. Y.; Meneguzzi, F.; Cardoso, R. C. “BDI agents in natural language environments”. In: Proceedings of the International Conference on Autonomous Agents and Multiagent Systems, 2024, pp. 880–888.

[33] Ichida, A. Y.; Meneguzzi, F.; Ruiz, D. D. “Measuring semantic similarity between sentences using a siamese neural network”. In: Proceedings of the International Joint Conference on Neural Network, 2018, pp. 1–7.

[34] Ioffe, S.; Szegedy, C. “Batch normalization: Accelerating deep network training by reducing internal covariate shift”. In: Proceedings of the International Conference on Machine Learning, 2015, pp. 448–456.

[35] Ji, Z.; Lee, N.; Frieske, R.; Yu, T.; Su, D.; Xu, Y.; Ishii, E.; Bang, Y. J.; Madotto, A.; Fung, P. “Survey of hallucination in natural language generation”, *ACM computing surveys*, vol. 55–12, mar 2023, pp. 1–38.

[36] Kahneman, D. “Thinking, fast and slow”. Macmillan, 2011, 512p.

[37] Lakoff, G. “Linguistics and natural logic”, *Synthese*, vol. 22–1, Dec 1970, pp. 151–271.

[38] Lin, B. Y.; Fu, Y.; Yang, K.; Brahman, F.; Huang, S.; Bhagavatula, C.; Ammanabrolu, P.; Choi, Y.; Ren, X. “Swiftsage: A generative agent with fast and slow thinking for complex interactive tasks”. In: Proceedings of the Advances in Neural Information Processing Systems, 2024, pp. 23813–23825.

[39] Liu, Y.; Ott, M.; Goyal, N.; Du, J.; Joshi, M.; Chen, D.; Levy, O.; Lewis, M.; Zettlemoyer, L.; Stoyanov, V. “Roberta: A robustly optimized bert pretraining approach”, *arXiv*, vol. abs/1907.11692–arXiv:1907.11692 [cs.CL], jul 2019, pp. 1–27.

[40] Maccartney, B. “Natural language inference”, Ph.D. Thesis, Stanford University, Stanford, CA, USA, 2009, 165p.

[41] MacCartney, B.; Manning, C. D. “An extended model of natural logic”. In: Proceedings of the International Conference on Computational Semantics, 2009, pp. 140–156.

[42] McCoy, T.; Pavlick, E.; Linzen, T. “Right for the wrong reasons: Diagnosing syntactic heuristics in natural language inference”. In: Proceedings of the Annual Meeting of the Association for Computational Linguistics, 2019, pp. 3428–3448.

[43] Meneguzzi, F.; de Silva, L. “Planning in bdi agents: a survey of the integration of planning algorithms and agent reasoning”, *Knowledge Engineering Review*, vol. 30–1, 1 2015, pp. 1–44.

[44] Mitchell, T. “Machine Learning”. McGraw-Hill, 1997, 414p.

[45] Montes, N.; Luck, M.; Osman, N.; Rodrigues, O.; Sierra, C. “Combining theory of mind and abductive reasoning in agent-oriented programming”, *Autonomous Agents and Multi-Agent Systems*, vol. 37–2, aug 2023, pp. 36.

[46] Moss, L. S. “Natural Logic”. John Wiley & Sons, Ltd, 2015, chap. 18, pp. 559–592, https://onlinelibrary.wiley.com/doi/pdf/10.1002/9781118882139.ch18.

[47] Murphy, K. P. “Machine Learning: A Probabilistic Perspective”. The MIT Press, 2012, 1104p.

[48] Nestorovic, T.; Matouek, V. “Modified conversational agent architecture”. In: IEEE 25th International Conference on Tools with Artificial Intelligence, 2013, pp. 682–689.

[49] Nguyen, A.; Wobcke, W. “An agent-based approach to dialogue management in personal assistants”. In: Proceedings of the International Conference on Intelligent User Interfaces, 2005, pp. 137–144.

[50] Nguyen, A.; Wobcke, W. “Extensibility and reuse in an agent-based dialogue model”. In: IEEE/WIC/ACM International Conference on Web Intelligence and Intelligent Agent Technology Workshops, 2006, pp. 367–371.

[51] Nye, M.; Tessler, M.; Tenenbaum, J.; Lake, B. M. “Improving coherence and consistency in neural sequence models with dual-system, neuro-symbolic reasoning”. In: Proceedings of the Advances in Neural Information Processing Systems, 2021, pp. 25192–25204.

[52] Qian, P.; Unhelkar, V. “Evaluating the role of interactivity on improving transparency in autonomous agents”. In: Proceedings of the International Conference on Autonomous Agents and Multiagent Systems, 2022, pp. 10831091.

[53] Qiu, X.; Sun, T.; Xu, Y.; Shao, Y.; Dai, N.; Huang, X. “Pre-trained models for natural language processing: A survey”, *Science China Technological Sciences*, vol. 63–10, sep 2020, pp. 1872–1897.

[54] Raffel, C.; Shazeer, N.; Roberts, A.; Lee, K.; Narang, S.; Matena, M.; Zhou, Y.; Li, W.; Liu, P. J. “Exploring the limits of transfer learning with a unified text-to-text transformer”, *The Journal of Machine Learning Research*, vol. 21–1, jan 2020, pp. 5485–5551.

[55] Rao, A. S. “Agentspeak (l): Bdi agents speak out in a logical computable language”. In: European workshop on modelling autonomous agents in a multi-agent world, 1996, pp. 42–55.

[56] Rao, A. S. “AgentSpeak(L): BDI agents speak out in a logical computable language”. In: Proceedings of the European Workshop on Modelling Autonomous Agents in a Multi-agent World, 1996, pp. 42–55.

[57] Richardson, K.; Hu, H.; Moss, L.; Sabharwal, A. “Probing natural language inference models through semantic fragments”. In: Proceedings of the AAAI conference on artificial intelligence, 2020, pp. 8713–8721.

[58] Rudin, C.; Radin, J. “Why are we using black box models in ai when we don’t need to? a lesson from an explainable ai competition”, *Harvard Data Science Review*, vol. 1–2, nov 2019, pp. 1–9.

[59] Russell, S.; Norvig, P. “Artificial Intelligence: A Modern Approach”. Prentice Hall Press, 2009, 1152p.

[60] Sado, F.; Loo, C. K.; Liew, W. S.; Kerzel, M.; Wermter, S. “Explainable goal-driven agents and robots-a comprehensive review”, *ACM Computing Surveys*, vol. 55–10, feb 2023, pp. 1–41.

[61] Sardina, S.; De Silva, L.; Padgham, L. “Hierarchical planning in bdi agent programming languages: A formal approach”. In: Proceedings of the International Joint Conference on Autonomous Agents and Multiagent Systems, 2006, pp. 1001–1008.

[62] Schaul, T.; Quan, J.; Antonoglou, I.; Silver, D. “Prioritized experience replay”. In: International Conference on Learning Representations, 2016, pp. 1–9.

[63] Schut, M.; Wooldridge, M.; Parsons, S. “On partially observable mdps and bdi models”. In: *Foundations and Applications of Multi-Agent Systems*, Springer, 2002, pp. 243–259.

[64] Shanahan, M. “Solving the frame problem: a mathematical investigation of the common sense law of inertia”. MIT press, 1997, 407p.

[65] Singh, D.; Hindriks, K. V. “Learning to improve agent behaviours in goal”. In: Programming Multi-Agent Systems: 10th International Workshop, ProMAS, 2013, pp. 158–173.

[66] Singh, D.; Sardina, S.; Padgham, L.; Airiau, S. e. p. “Learning context conditions for bdi plan selection”. In: Proceedings of the International Conference on Autonomous Agents and Multiagent Systems, 2010, pp. 325–332.

[67] Singh, D.; Sardina, S.; Padgham, L.; James, G. “Integrating learning into a bdi agent for environments with changing dynamics”. In: Proceedings of the International Joint Conference on Artificial Intelligence, 2011, pp. 1–6.

[68] Srivastava, S.; Labutov, I.; Mitchell, T. “Zero-shot learning of classifiers from natural language quantification”. In: Proceedings of the Annual Meeting of the Association for Computational Linguistics, 2018, pp. 306–316.

[69] Stechly, K.; Valmeekam, K.; Kambhampati, S. “Chain of thoughtlessness? an analysis of cot in planning”. In: Proceedings of the Advances in Neural Information Processing Systems, 2025, pp. 29106–29141.

[70] Sutskever, I.; Vinyals, O.; Le, Q. V. “Sequence to sequence learning with neural networks”. In: Proceedings of the Advances in Neural Information Processing Systems, 2014, pp. 3104–3112.

[71] Sutton, R. S.; Barto, A. G. “Reinforcement learning: An introduction”. MIT press, 2018, 552p.

[72] Tafjord, O.; Dalvi, B.; Clark, P. “ProofWriter: Generating implications, proofs, and abductive statements over natural language”. In: Findings of the Association for Computational Linguistics, 2021, pp. 3621–3634.

[73] Torabi, F.; Warnell, G.; Stone, P. “Behavioral cloning from observation”. In: Proceedings of the International Joint Conference on Artificial Intelligence, 2018, pp. 4950–4957.

[74] Valencia, V. S. “Studies on natural logic and categorial grammar”, Ph.D. Thesis, Universiteit van Amsterdam, 1991, 197p.

[75] Valmeekam, K.; Marquez, M.; Olmo, A.; Sreedharan, S.; Kambhampati, S. “Planbench: An extensible benchmark for evaluating large language models on planning and reasoning about change”. In: Proceedings of the Advances in Neural Information Processing Systems, 2023, pp. 38975–38987.

[76] Valmeekam, K.; Olmo, A.; Sreedharan, S.; Kambhampati, S. “Large language models still can’t plan (a benchmark for LLMs on planning and reasoning about change)”. In: NeurIPS Foundation Models for Decision Making Workshop, 2022, pp. 1–21.

[77] Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, Ł.; Polosukhin, I. “Attention is all you need”. In: Proceedings of the Advances in Neural Information Processing Systems, 2017, pp. 5998–6008.

[78] Verhagen, R. S.; Neerincx, M. A.; Tielman, M. L. “A two-dimensional explanation framework to classify ai as incomprehensible, interpretable, or understandable”. In: International Workshop on Explainable, Transparent Autonomous Agents and Multi-Agent Systems, 2021, pp. 119–138.

[79] Verma, M.; Bhambri, S.; Kambhampati, S. “Theory of mind abilities of large language models in human-robot interaction: An illusion?” In: Companion of the ACM/IEEE International Conference on Human-Robot Interaction, 2024, pp. 36–45.

[80] Wang, R.; Jansen, P.; Côté, M.-A.; Ammanabrolu, P. “ScienceWorld: Is your agent smarter than a 5th grader?” In: Proceedings of the Conference on Empirical Methods in Natural Language Processing, 2022, pp. 11279–11298.

[81] Wang, W.; Wei, F.; Dong, L.; Bao, H.; Yang, N.; Zhou, M. “Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers”. In: Proceedings of the Advances in Neural Information Processing Systems, 2020, pp. 5776–5788.

[82] Weiss, G. “Multiagent Systems”. MIT Press, 2013, 920p.

[83] Williams, A.; Nangia, N.; Bowman, S. “A broad-coverage challenge corpus for sentence understanding through inference”. In: Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2018, pp. 1112–1122.

[84] Winikoff, M.; Padgham, L.; Harland, J.; Thangarajah, J. “Declarative & procedural goals in intelligent agent systems”. In: Proceedings of the International Conference on Principles of Knowledge Representation and Reasoning, 2002, pp. 470–481.

[85] Winikoff, M.; Sidorenko, G. “Evaluating a mechanism for explaining bdi agent behaviour”. In: International Workshop on Explainable, Transparent Autonomous Agents and Multi-Agent Systems, 2023, pp. 18–37.

[86] Winikoff, M.; Sidorenko, G.; Dignum, V.; Dignum, F. “Why bad coffee? explaining bdi agent behaviour with valuings”, *Artificial Intelligence*, vol. 300, nov 2021, pp. 103554.

[87] Wolf, T.; Debut, L.; Sanh, V.; Chaumond, J.; Delangue, C.; Moi, A.; Cistac, P.; Rault, T.; Louf, R.; Funtowicz, M.; Davison, J.; Shleifer, S.; von Platen, P.; Ma, C.; Jernite, Y.; Plu, J.; Xu, C.; Scao, T. L.; Gugger, S.; Drame, M.; Lhoest, Q.; Rush, A. M. “Transformers: State-of-the-art natural language processing”. In: Proceedings of the Conference on Empirical Methods in Natural Language Processing: System Demonstrations, 2020, pp. 38–45.

[88] Wong, W.; Cavedon, L.; Thangarajah, J.; Padgham, L. “Flexible conversation management using a BDI agent approach”. In: Intelligent Virtual Agents - International Conference, 2012, pp. 464–470.

[89] Wu, T.; He, S.; Liu, J.; Sun, S.; Liu, K.; Han, Q.-L.; Tang, Y. “A brief overview of chatgpt: The history, status quo and potential future development”, *IEEE/CAA Journal of Automatica Sinica*, vol. 10–5, may 2023, pp. 1122–1136.

APPENDIX A – NATURAL LANGUAGE PLANS FOR SCIENCEWORLD

This appendix reports natural language plans we develop to conduct our experiments in Chapter 4. First, we show the plans for the ScienceWorld "melt" task to deal with an example of test variation. Second, we report an example of a plan for the task "find-non-living-thing" to illustrate how we approach it using natural language plans.

A.1 Melt Task

In this section, we report a set of natural language plans to handle the test variation "melt mercury" in Listing A.1. In the ScienceWorld environment, as the sequence of steps for melting mercury contains a long sequence of steps, we develop plans in a hierarchical way by generating goal additions (i.e., subgoals), and evaluating the plan selection mechanism to trigger other plans. To develop this plan, we leverage the trajectories provided by Wang *et al.* [80] that address the variation.

The first plan in Listing A.1 represents the main plan, which is initially triggered when the agent perceives the task description. The plan body of this plan calls 5 other plans by including goal addition events in the NatBDI's event queue. Recall from Section 4.3 that we generate plans automatically for move the agent to any room and, thus, we abstract plans such as "move to kitchen" assuming that the plans is already in the plan library.

Listing A.1 – All natural language plans to melt mercury in the ScienceWorld environment

```
IF your goal is to melt mercury THEN
    PLAN TO move to kitchen,
    pick up thermometer,
    PLAN TO get metal pot,
    PLAN TO get mercury,
    focus on substance in metal pot,
    PLAN TO refrigerate mercury,
    PLAN TO heat mercury on blast furnace,
    focus on substance in metal pot
—
IF your goal is to get mercury
CONSIDERING you are not in workshop AND you have metal pot in your inventory
THEN
    PLAN TO move to workshop,
    pour glass cup into metal pot in inventory
—
```

```
IF your goal is to refrigerate mercury
CONSIDERING you have a metal pot with mercury in your inventory
THEN
    PLAN TO move to workshop,
    open freezer,
    move metal pot to freezer,
    wait,
    wait,
    use thermometer on metal pot,
    focus on substance in metal pot,
    pick up metal pot
```

—

```
IF your goal is to heat mercury on blast furnace
CONSIDERING you have metal pot with mercury in your inventory
THEN
    PLAN TO move to foundry,
    open blast furnace,
    move metal pot to blast furnace,
    activate blast furnace,
    use thermometer on metal pot,
    focus on substance in metal pot,
    wait,
    use thermometer on metal pot
```

A.2 Find-non-living-thing Task

In this section, we report an example of a natural language plan to deal with the “find-non-living-thing” task, which is the simplest task in our experiments. Although this task has 75 test variations, the only aspects that changes between variations are the agent’s starting location and the target container specified from task description. Thus, we only show a single example to illustrate how we develop plans to this task in Listing A.2.

Compared to the task in Appendix A.1, the plan from A.2 consists of a short sequence of steps, and we not include any subplans to address the task. Since the task requires picking any non-living thing, we could include an action to pick an object other than the thermometer. Finally, the plan moves the agent to the target location, as specified in the task description, and places the object in the target container.

Listing A.2 – Natural language plan to retrieve a non-living thing and place it in the orange box located in living room.

```
IF your goal is to find a(n) non—living thing by moving it to the orange box
in the living room
THEN
    PLAN TO move to kitchen,
    pick up thermometer,
    focus on thermometer,
    PLAN TO move to living room,
    move thermometer to orange box
```

Pontifícia Universidade Católica do Rio Grande do Sul
Pró-Reitoria de Pesquisa e Pós-Graduação
Av. Ipiranga, 6681 – Prédio 1 – Térreo
Porto Alegre – RS – Brasil
Fone: (51) 3320-3513
E-mail: propesq@pucrs.br
Site: www.pucrs.br